Unity Package for Abstracting the Development Process of Games with a GitHub/GitLab UI

by Helen Pui-Sarn Chang

Dissertation

Submitted in partial fulfilment for the degree of
MEng Software Engineering

Supervised by Dr. Manuel Maarek

Heriot-Watt University
School of Mathematical and Computer Sciences

9th April 2026

AUTHORSHIP DECLARATION

Course code and name	F20PA Research Methods and Requirements Engineering
Type of assessment	Individual
Coursework title	Dissertation (D3)
Student name	Helen Pui-Sarn Chang
Student ID	H00401005

By signing this form:

- I declare that the work I have submitted for individual assessment or the work I have contributed to a group assessment is entirely my own. I have not taken the ideas, writings or inventions of another person and used them as if they were my own. My submission or my contribution to a group submission is expressed in my own words. Any uses made within this work of the ideas, writings or inventions of others, or of any existing sources of information (books, journals, websites, etc.) are properly acknowledged and listed in the references and/or acknowledgements sections.
- I confirm that I have read, understood and followed the University's Regulations on academic misconduct as published on the University's website, and that I am aware of the penalties that I will face should I not adhere to the University Regulations.
- I confirm that I have read, understood and avoided the different types of plagiarism explained in the University guidance on Academic Integrity and Plagiarism.

Student signature: Helen Pui-Sarn Chang

Date: 9th April 2026

Your work will not be marked if a signed copy of this form is not included with your submission.

ACKNOWLEDGEMENTS

I would like to give my thanks to my supervisor, Dr. Manuel Maarek, for his guidance and patience throughout this project.
I would also like to give my thanks to my family for their continued support throughout my time at university.

ABSTRACT

There has been an interest in games that use GitHub/GitLab as an interface, with a substantial number of these being serious games involving coding exercises. To ease the development process of these games, this project aims to produce a Unity Package which abstracts the code that handles interactions with the GitLab Representational State Transfer (REST) Application Programming Interface (API) in a way that facilitates the creation of GitLab User interface (UI) games.

To achieve this, an exploration into literature about code as a game interaction (with a majority discussing serious games), good API design and input validation was required. A look into the GitLab REST API, as well as the design and execution of an experiment looking into the abstract method call preferences of developers, was also essential.

TABLE OF CONTENTS

1 Introduction

In this section, we introduce our goals and motivations for making a GitHub/GitLab User Interface (UI) game development package, what sub-goals we have to achieve that goal, and the organisation of the rest of the paper.

1.1 Background and Motivation

Interest in the field of Computer Science has increased in recent years, with UCAS (2023) saying there was an increase of 33.3% in computing applications from 2021 to 2023, and The British Computer Society (2025) declaring that the number of Computing degree applications increased in the previous 5 years, with Computing ranking 7th in popularity amongst the UK 18-year-old demographic. Fay (2025) reports a demand for prospective tech workforce employees to have prior experience, and how AI hinders their ability to gain experience by replacing roles. Additionally, Dohmke (2023) announced that in 2023, GitHub surpassed its goal of having over 1 million developers using its service. This tool is widely used; therefore, having experience with it prior to entering any workforce or working on anything professional is very likely to be beneficial to students. We therefore believe that modifying the way people are taught how to code to be more like the workplace and industry makes the education they receive more valuable and worthwhile.

As of recent, there has been a rise in games using coding in their interactions (Min *et al*, 2020; Rojas and Fraser, 2016; Korkiakoski *et al.*, 2023) or even using GitHub/GitLab as an interface, such as the game platform "Citadel Programming Lab" by McGregor *et al.* (2022) and Maarek *et al.* (2019). In addition, Zhan *et al.* (2022, pp. 8-9) had results that showed gamification's capability in aiding the learning of programming. Their results showed that gamification had much better results when supporting text-based learning as compared to when supporting graphical learning.

The experience to be gained through interaction with a GitHub/GitLab UI, along with the benefits from gamification, is why we believe that GitHub/GitLab UI games are important to help better prepare students for the workforce. Despite this, however, the number of GitHub/GitLab UI games are low in number, with serious games teaching programming preferring to implement their own UI (Bishop et al., 2015; Feldmeier, Straubinger and Fraser, 2023; Rojas and Fraser, 2016) - whilst still beneficial, these lack the workplace-relevant experience that having a GitHub/GitLab UI would provide. We have opted to focus on the GitLab Application Programming Interface (API) due to GitLab being open-source and more likely to be used by students.

1.2 Aim and Objectives

We aim to create a Unity package that simplifies and facilitates the creation of games that use GitLab as a user interface, so that GitLab UI games will become a more popular computer science serious games genre. Through this, we hope to increase the number of these games so that they may reach and provide a wider audience with a more gamified and industry-relevant experience during their time learning how to program.

- Establish requirements for our Unity package through analysis of literature.
- Design and implement, without our Unity package, a set of example games that showcase the capabilities we want our Unity Package to have.

- Conduct an experiment in a controlled setting using questionnaires to ascertain stakeholders' preferred API method call styles for our package. This will make use of code from our example games.
- Established the core requirements for our Unity package through analysis of our experiment results.
- Implement our Unity package as well as proper documentation for it using the Scrum framework, as we need to ensure we have a functional version of our package to be able to test its functionality. Using Scrum will ensure we have functional versions of our code at the end of each sprint.
- Re-implement our example games with the use of our Unity Package and analyse the effect our package had on the games' code.

Our work should help propagate GitLab UI games further in the educational sectors of the programming community, therefore helping programming education improve through the benefits of gamification and industry-relevant experience.

1.3 Organisation

In section 2, we explore papers on the definition of serious games and what categories of computer science serious games there are, before looking deeper into papers about what types of games these serious games fall into. We then look at API guidelines and security standards. From those papers, we try to derive the most important capabilities that we should try to recreate in our Unity package. In section 3, we discuss our goals, how we plan to achieve them and how we plan to evaluate them. In section 4, we look into proving our project is feasible by looking into the design of our project, any setup we have done in terms of coding environment, and discussing the prototype we have created for the project.

Now that we have gone over our motivations and had a high-level look at what tasks we need to achieve, we look deeper into the topic by diving into relevant literature.

2 Background Research

In this section, we will explore five concepts surrounding the needs of a GitLab UI game development package. In section 2.1, we explore what serious games are and what gets taught using serious games in computer science. In section 2.2, we explore patterns in cybersecurity serious games. In section 2.3, we look at trends in general computer science serious games. In section 2.4, we look at what makes a good API design. In section 2.5, we look at a security standard and analyse what could make our package safer. In section 2.6, we conclude and summarise the entire section.

2.1 Serious Games

Serious games lack a concrete definition, and the definitions we have currently differ greatly – this is discussed by Laamarti, Eid and El Saddik (2014, pp. 2-4), who explain a variety of different classifications of the meaning of Serious Games. These meanings all differ in some way, such as by having different numbers of criteria or one specifying the need for the game to be computer software, whilst another instead declares the age group of the target audience an important factor. They then go on to describe their own classification for what constitutes a serious game created after going through a variety of literature on the topic, they state that: a serious game must be an application that has a dimension of entertainment; contain not just one type of media such as graphics, animation, or audio but two or more; and must be able to convey some knowledge to the

player. They additionally state that the most popular definition is that by Michael and Chen (2005, quoted in Laamarti, Eid and El Saddik, 2014, p. 3), whose definition of serious games has only one requirement of being a game with a main objective of anything except amusing players.

Now that we know what a serious game is, we can look at serious games in education. Luxton-Reilly *et al.* (2018, pp. 64,69,77) claim that the use of gamification is not a new topic of interest in computer science. They note a rise in papers which explore how gamification can be used to benefit introductory programming in the time period from 2003 to 2017. They comment that games are a popular motivation method in the field of introductory programming and that learning aid educational games were popular amongst researchers in the time period they reviewed. They then discuss the ways that digital games have been used in instructing people on software development, such as by simplifying the development process of games. They then look over several reviews investigating computer-science-related serious games. One such review went over 100 games, a large proportion of which taught the foundational concepts of how to develop software. Through this paper, we can infer that serious games are a prevalent topic in Computer Science education during the past and present and that games facilitating the learning of software development are one of the more popular categories in computer science serious games.

A topic in computer science that particularly uses serious games is cybersecurity. Zhang-Kennedy and Chiasson (2021, pp. 5-6) claims that digital games are one of the most common tool types used in teaching cybersecurity with it making up more than a third of the cybersecurity teaching tools that they had found – these games spread across a plethora of genres which includes quiz games, puzzle games and serious games that aim to educate people on cybersecurity by focusing on smaller sub-topics to teach such as the identification of phishing attacks in URLs. They also suggested that, regardless of age, digital educational games were the dominant tools used to teach cybersecurity. We can deduce that serious games are a favoured method to teach cybersecurity and that digital games are a common type of game within cybersecurity serious games.

2.2 Cybersecurity-based Games

Focusing on cybersecurity-based games, we look at what games already exist and what kind of gameplay people will likely want to achieve. Ng and Hasan (2025, p. 7) describe the range of serious games that can be found in the variety of cybersecurity-focused serious games they looked at and categorise these games into subsections. Quiz games which focus on teaching through question-and-answer tasks. Card, board and tower defence games, which utilise attack and defence metaphors to make the concepts easier to understand. Simulation games, which mostly require players to have prior knowledge of cybersecurity, provide a more accurate experience. Escape room games and puzzle games, which mainly rely on their puzzles and tasks to keep players engaged in learning. Casual games utilise their short and simple nature to make playing through and completing the experience easy. Narrative games (which overlap with many of the previously stated categories) and story-driven puzzle games both follow a storyline and impress upon players the impact and importance of cybersecurity techniques through the narrative being told in the game. Among these types, quiz-based gamification was stated to be the most popular genre as it made up 35.8% of the games. Although most of these will be passing strings like passwords or card options through files in GitLab repositories, special cases may require more complex functionality in the user interface. Card and board games, puzzle games, escape room games, and simulation games may all require

you to change the position of files in a repository to other directories; therefore, our package should be able to detect this. Escape room games and simulation games most likely require that certain files be left untouched by the player, so a mechanism to check for changes may be desirable.

Given the nature of GitLab being closely connected to code and computation, one of the most probable types of cybersecurity games requiring the use of a GitLab UI would be one involving the use of code to implement cybersecurity concepts. A good example of this is the cybersecurity tower defence serious game platform named "Citadel Programming Lab" which was co-created by McGregor *et al.* (2022, pp. 486-487) and Maarek *et al.* (2019). McGregor et al. (2022, pp. 488-489) describe the game's main game loop as playing the tutorial, playing the actual game itself and unlocking upgrades through completing programming tasks in their GitLab-based web interface. The type of tasks they can do to unlock upgrades are coding activities centred around accomplishing typical cybersecurity needs such as shortening URLs, encryption, and verification. Players submit their code for the task, then unit tests are used to decide whether the player has correctly completed the task or not. For our package to be capable of recreating this, it would need to be able to either retrieve the code files themselves or the test results from the GitLab repository. Another example is the cybersecurity augmented reality game called "Hack The Room" by Korkiakoski *et al.* (2023, pp. 350-352), which was implemented with the Unity game engine. This game had Linux terminals in the augmented reality environment for players to input commands into in order to move onto the next level. The most prominent shared feature in these games is that they both attempt to recreate a real coding space (these being a GitLab interface or a Linux terminal) – since our Unity Package is designed to facilitate a GitLab UI, and therefore provides players with experience in using the GitLab interface intrinsically, our package is guaranteed to provide this quality to any games developed with it.

2.3 General Computing-based Games

Expanding our search to the more generalised area of computer science serious games, we look at the categories that these games fall into. Types of educational games used to gamify the study of computer science are discussed by Videnovik *et al.* (2023, pp. 10-11). They describe a variety of computer science serious games with puzzle games, quiz games, escape room games, and block-based environment games being the most prominent examples. Additionally, they note that primary school students are the most common target participants for game creation experiences. From this paper we can assume a higher likelihood of puzzle games, quiz games and escape room games being created using our package (with block-based games being unviable in a GitLab UI) and accordingly place more focus on requirements stemming from these genres – the most important capability these three types of games may require is the ability to display things that simple images in a file cannot recreate such as displaying puzzles of different types or displaying a quiz in a more engaging style. However, due to one main goal of our Unity Package being to ensure players gain proper experience with GitLab, we should avoid changing the way the GitLab display looks and should constrain our package's "display" to normal GitLab activity, such as through file names, file contents, and folder names as to retain the normal GitLab interface. Therefore, the second-best solution to this requirement in a GitLab-UI-using context is the ability to format and to check the format of text within files (for example, extra spaces or ensuring text starts on a new line). Not only could quiz games use the formatting to add clarity and style to their question sheets, but formatting would also allow our Unity Package to be used to create a more flexible

variety of puzzles for puzzle games and escape room games. As younger children are the most common target audience for game-creation experiences, we should again ensure our package can revert unwanted changes to prevent frustration.

Given the nature of GitLab being closely connected to code and computation, one of the most probable use cases for why developers would choose to use GitLab as a user interface would be to create games that incorporate coding in some way into their gameplay. Min *et al.* (2020, p. 655) present "ENGAGE", a narrative-driven game where the player must go through a story while using block-based coding to complete tasks such as making an algorithm to brute-force a password, making a program to direct a quadcopter's flight path, and programming a crane to move objects around. Recreating such capabilities in our package would require the ability for our package to retrieve code files from a GitLab repository. A game created by Bishop et al. (2015, pp. 399-400) named "Code Hunt" is centred around a series of puzzles that players must create an algorithm to solve in the built-in editor window . Once the player tests their code, the player either wins, receives a list of mismatches between their algorithm and the goal solution algorithm, or receives a complication error. Code Hunt was used in several contests, which involved many calls to their personal Code Hunt back-end Representational State Transfer (REST)-based service API from players using the front-end cloud application. If our Unity package were to attempt to facilitate such activity, it could imitate the several different instances of Code Hunt for the players by having a separate repository or separate files for each player to submit their code to. It would also need to be able to retrieve code files and return the results of a success, error, or a list of mismatches, which all have the capability of being text and therefore would only require our package to be able to upload text to a normal or markdown file to the GitLab repository.

Some more good examples of coding games are games that focus on creating variants of a source code and creating tests to detect these variants. One example is the two-player mutation testing game named "Code Defenders" by Rojas, J.M. and Fraser, G. (2016, pp. 163-164) in which one player creates variations of a target program by editing its source code while the other player creates code for unit tests aimed to detect variants of the target program. The player creating variant programs must create a unit test countering their own program if challenged by the other player. The player writing tests can either receive a `diff` report as hints on easy mode or descriptions of file changes on hard mode. In terms of recreation in our package, the hints for hard mode are easy due to them simply being text. However, the `diff` report may need to be converted to a text or markdown file. Our package should be able to access multiple repositories of different protection levels so we can conceal workspaces from other players. As with the other games, our package would need to be able to retrieve file text from a repository to run code. Although it is a gamification plugin and not a game, it has a relevant user interface style – this tool, called "Gamekins", was created by Straubinger and Fraser (2022, pp. 85-88). It is a Continuous Integration (CI) environment plugin for the Jenkins CI environment, designed to gamify software testing. Jenkins is triggered to start Gamekins when code is committed by a developer to the version control system it's connected to. There are six types of challenges in Gamekins which are tasks assigned to developers for them to complete, the challenge types ask developers to: fix failed builds; write one more test without narrowing the area of code the test targets; improve the project's ability to detect variants of the code by adding more intricacy to the project's tests; removing parts of the project's tests or code that were deemed unfavourable during analysis by Gamekins; and provide more code coverage (which Codecademy Team (2025) states is the percentage of a chosen

collection of code that is executed when the code's associated automated tests are run) either to a class, method, or a line of code. When replacing completed challenges, classes with lower code coverage and the most recent update time are prioritised. Quests are collections of challenges that are completed one at a time and only generate predetermined types of challenges. Gamekins has leaderboards, a selection of avatars for these leaderboards, and achievements to be earned. Challenges and quests can be rejected. The main capabilities our Unity Package would require for recreation of these behaviours is again the ability to retrieve files so that we may retrieve tests and the code to be tested, the ability to upload to a text file or markdown file the current status of different challenges and quests as well as display a leaderboard, the capability of retrieving the update time of files for challenge creation priority, and the ability to be able to see the results of testing once a developer has pushed a new commit.

2.4 API Guidelines

In this section, we look at guidance for how APIs should be designed for ease of use – this is relevant as our Unity package aims to abstract the GitLab API and make it easier to use, therefore, making it applicable to these guidelines. Rama and Kak (2015, pp. 77-82) describe nine features of APIs that they claim are commonly considered frustrating for developers. API methods having different return values but similar or identical names will cause developers to have to memorise method call return types. API methods having multiple parameters of the same type cause developers to have to keep referring to the documentation - methods having too many parameters, regardless of type, also cause this issue. API methods that ask for similar parameters in different orders, which could cause developers to pass parameters in the wrong order. APIs having an abundance of methods with similar names, causing developers to have to refer to documentation to find out the differences between the methods (especially in the case of the methods performing similar tasks). APIs not describing similar methods together in their documentation makes finding the best method for a task harder. API documentation that does not mention its methods' thread-safety status. APIs that throw undescriptive exceptions. API with poor quality documentation.

Distancing ourselves from what we should not be doing and focusing on what we should be doing, we look at some guidelines. A set of eight general guidelines is described by Henning (2009, pp. 51-55). APIs must be able to perform the function for which it was designed - for example, if they are designed to wait, then there should not be an upper boundary to the length of time they can wait. APIs should be minimised in terms of having fewer function calls, parameters, and types, when possible, but still include extra methods, provided they will be commonly used. API design choices must be made with the API's use context in mind. APIs should have sufficient detail in their design so that it is general enough to be applicable to all relevant use cases but specific enough to catch as many errors as possible before runtime to prevent frustration. APIs should have user-centred designs, which can be achieved through activities such as considering what an API's methods look like from their perspective or asking them for a desired method call directly. APIs should try to lessen the workload of calling its methods on developers by ensuring any work the user developer would have to do to call the API method is moved into the API itself when at all possible - this also includes narrowing down the capabilities of the API methods to decrease features that add extra work for developers such as method flag parameters which add extra functionality to the method depending on if it is true or false but also means developers must choose values for the flag every time the function is called (the loss of these extra parameters can be compensated for by ensuring the methods provided can be combined together to form these more complex features). API

methods and documentation should be written prior to implementation, with the documentation being complete to avoid developers not knowing details that they may need. (They recommend the methods being largely influenced by prospective users through discussing with them the method calls they would prefer to use for this API, and by having new users try out the API to bring a new perspective.) API methods should be efficient to use – mainly, they should share similarities with other methods from the same API and other APIs, as this makes the methods easier to learn and use. An additional detail they mention is that it is good for any returned error values to have options other than strings, as although strings provide readability, they make it difficult to use for decision-making in programs.

Myers and Stylos (2016, p. 67) present a set of ten API guidelines created by applying a pre-existing set of heuristic evaluation guidelines to API design. API state should be easy to view, and API operations should return proper feedback. API method and class names should be the most expected and intuitive option. APIs should be easily reset to a default state so that any complex series of actions won't frustrate anyone trying to revert them. API design should be consistent. API design should do as much as possible to guide developers away from errors, such as by having default methods that work for the most common use cases. API naming conventions should have easily recognisable meanings to avoid having to look at documentation, but differ enough to avoid confusion. APIs should be able to perform developers' requests efficiently. API designs should be minimalised. APIs should provide helpful error messages to help developers debug their code. APIs must have proper documentation.

Overall, the most common guidelines seem to be: minimalize methods; have easily differentiable but intuitive method and parameter names; methods should have similar names, parameter names, and parameter order with methods that have similar functionality; APIs should be well documented (especially in terms of thread safety); APIs should have easily understood feedback; and APIs should balance between preventing user error but allowing user flexibility. Other important but less common guidelines are: Error messages should be returned in several forms; an API's current state should be easily visible; and APIs should be easily reset to a default state. The most important guideline is that API design should be heavily influenced by users through gathering info from them or perceiving it from their point of view.

2.5 Security

Since our Unity Package will be taking input from GitLab and oftentimes in the form of text from users, we need to do input validation on the input. Open Worldwide Application Security Project (OWASP, 2025, pp. 24-28, 31, 40, 91) provides an Application Security Verification standard. Although a few guidelines are relevant to the games created using our package, most pertain to running code, which our package does not do. There are also several guidelines relevant to the package itself. Any error messages returned to end-users must not contain any sensitive information (since our users are developers, this does not apply to us). Input should be checked before being processed to confirm it is of the correct form, then decoded once, and should not be decoded again. Output should be encoded only right before being used and not before that. Untrusted data must be encoded when building URLs for use, such as when a game using our package asks an end-user to input repository details for use. We should ensure resources are released once no longer needed. Parsing of data and encoding of output characters must be done using the same techniques (differing only between data types) throughout the program. Anti-automation controls are required to prevent excessive calls by ensuring interactions are human-like. We must

avoid creating Uniform Resource Identifiers (URIs) or header fields that go into Hypertext Transfer Protocol (HTTP) requests which are too long to be accepted by the message target, as these could cause a continuous stream of error responses. We must ensure our package fails and handles errors with grace.

2.6 Summary

Throughout the last 4 sub-sections, we have looked at a variety of computing and cybersecurity serious games and what typical functionality they have, and we have picked out the most important functionalities for our Unity package to have the capability to recreate. We have also looked at API design guidelines and an application security standard that we should implement rules from to make our package easier and safer to use. Overall, we have gleaned a great deal of information on what our package should have so that it may offer good usability to users. We will now move on to section 3, where we discuss what our main goal is and our overall plan, requirements, and tasks towards reaching that goal as well as our evaluation plan for the final product.

3 Work Plan

In this section, we will discuss what the project's main goal is and how we will reach that goal. In section 3.1, we discuss the actual goal itself. In section 3.2, we discuss and prioritise the requirements of our project. In section 3.3, we discuss the tasks we need to achieve to accomplish the goal from section 3.1. In section 3.4, we discuss how to evaluate our final product at the end.

3.1 Research Goals

For the purpose of creating a Unity package that will truly be able to help developers create games employing the use of a GitLab UI, we have to discover the ways in which packages best assist developers and how to minimise frustration when using our package so that developers will choose to use our package rather than resorting to accessing the GitLab API by themselves. In short, we have one research question: *What features should an optimised GitLab user-interface game creation package have?*

3.2 Requirements

The goal of this project is then to find the features that an optimised GitLab UI game creation package should have and implement them into our Unity package. From this goal and the background research we conducted, we can identify a list of requirements for our Unity package. These will be split into Functional Requirements (FR) and Non-Functional Requirements (NFR) and will have titles in bold for the main sections. Additionally, the requirements will be prioritised with MoSCoW rankings, which are: Must, Should, Could, and Won't. The requirements are split into sections with a title in bold, followed by a description of the section, and then followed in italics by the reasoning for the MoSCoW rankings given to the requirements in that section.

ID	Requirement	Priority
FR-1	**Web Requests**: Users shall be able to successfully make HTTP requests to a GitLab host; *Rationale: Musts are core features that the package relies on, whilst the ones ranked Could are less vital to the package's functionality.*	
FR-1.1	Users shall be able to make requests to both the gitlab.com host and the gitlab-student.macs.hw.ac.uk host.	Must
FR-1.2	Users shall be able to retrieve data from GitLab.	Must

FR-1.2.1	Users shall be able to retrieve file names, folder names, file contents, and repository structure from a repository.	Must
FR-1.2.2	Users shall be able to retrieve unit test results.	Could
FR-1.2.3	Users shall be able to retrieve details from user profiles and the update time of files.	Could
FR-1.3	Users shall be able to send requests to GitLab along with data attached if required.	Must
FR-1.3.1	Users should be able to send multiple different requests without it causing an error due to too many HTTP requests being sent at the same time.	Must
FR-1.3.2	Users shall be able to create new files and folders in a repository or update the contents of pre-existing ones.	Must
FR-1.3.3	Users shall be able to delete files and folders from a repository.	Must
FR-1.4	Users shall be able to use variants of the HTTP request methods which encode extra parts of the HTTP requests where possible.	Must
FR-1.5	Users can interact with public, protected, and private repositories.	Must
FR-1.6	Users can interact with multiple repositories in the same script.	Must
FR-1.7	Users can format text in files through indentation, spacing, etc.	Must
FR-2	**Preservation**: Users shall be able to call methods which continually check for changes to files or changes to a repository's structure and will revert them; *Rationale: Musts are core parts of the functionality of Preservation, whilst Coulds only are alternative, more effective ways to implement Preservation's functionality..*	
FR-2.1	Users shall be able to check for changes to a repository or file.	Must
FR-2.1.1	Changes in a repository or file will be detected through continuous polling.	Must
FR-2.1.2	Changes in a repository or file will be detected through the use of webhooks.	Could
FR-2.1.2.1	Users shall be able to call a variant of the webhook methods that will send an error if changes are being made faster than humanly possible.	Could
FR-2.2	Users shall be able to start and end a function to delete extra folders and files in a repository.	Must
FR-2.3	Users shall be able to start and end a function to replace the contents of folders and files with contents provided by the user if they are changed.	Must
FR-3	**Transparency:** Users shall be able to call methods that make understanding or changing what's going on internally much easier; *Rationale: All of these are Could as they are not integral to the functionality or purpose of the package.*	
FR-3.1	Users shall be able to easily access the state of the HTTP request queue and whether or not certain HTTP requests have been sent or completed.	Should
FR-3.2	Users shall be able to use a method to reset the package to having no HTTP requests queued.	Should
NFR-1	**Usability:** The package will be easier to use than users implementing the functionality themselves; *Rationale: Everything is a Must as this is the main goal of the package.*	Must
NFR-1.1	The package will be capable of putting together a URL from parts given by the user.	Must
NFR-1.2	The package will decrease the number of lines in a program's code when used.	Must
NFR-2	The package should have an easily understood and intuitive set of method calls.	Must
NFR-2.1	The package should have method names that easily identify their use and are easily differentiable from other methods in the package.	Must
NFR-2.1.1	The package's methods and parameters should have similar names and order to the methods and parameters with similar functionality.	Should
NFR-2.2	The package's methods should have self-explanatory parameter names.	Must
NFR-2.2.1	The package's method parameter names should have a similar order and name style to those found in methods with similar functionality.	Must
NFR-2.3	The package's methods should return easily understood feedback for errors and successes.	Must

NFR-2.3.1	The package should return error information in the form of both a human-readable string and an integer.	Should
NFR-3	**Security:** The package shall adhere to OWASP security guidelines; *Rationale: Most are Musts as security is integral to the professionalism of the package, but the requirements ranked Won't are outside our scope as they apply to running code.*	Must
NFR-3.1	The package shall use the same techniques to parse and encode data (differing techniques may be used for different data types) throughout itself.	Must
NFR-3.2	The package shall check that user input (GitLab host name, project number, etc.) is of the correct type (string, integer, etc.)	Must
NFR-3.3	The package will be capable of restricting the size of URI/URL and header fields that go into HTTP requests.	Must
NFR-3.4	The package will only encode parts of HTTP requests right before they are sent.	Must
NFR-3.5	The package shall release resources when no longer needed.	Must
NFR-3.6	The package will perform input-sanitisation on code read from files before sending the code to the user to be executed, as well as warn the user if anything was detected.	Won't
NFR-3.6.1	The package will scan code files with an antivirus before passing them to the user.	Won't
NFR-4	**Documentation:** The package's documentation shall be complete; *Rationale: the Must is the core of the documentation, the Should is very important but not the basis of the documentation and the Could is information that won't apply to most users.*	
NFR-4.1	The package's documentation shall include information about the thread-safety of its methods.	Could
NFR-4.2	The package's documentation shall include information about what shall occur should the methods encounter an error.	Should
NFR-4.3	The package's documentation shall detail the package's parameters, how to use it, and return its values.	Must

3.3 Tasks

During the first semester, we discovered and read a wide variety of documentation on package creation, the definition of serious games, computer science serious games, cybersecurity serious games, API design guidelines, and security guidelines. We analysed the requirements and goals for our project and then proved the feasibility of our project by creating a prototype script in Unity that sends web requests that cover the main basic capabilities of our package. We then created a new script abstracting that code into callable functions. We have designed and created draft documents for an experiment looking into participants' preferred abstraction for a GitLab Unity package. In the second semester, we will have to achieve several things. We must create two example games without any abstraction and modify a pre-existing game to use GitLab as a UI. We must use code from our example games to finish creating and then perform the experiment we designed. Once we have analysed the results of the experiment, we must create our Unity package and its documentation. Then we adapt the three games from the beginning to use our package and analyse the effects of our package.

3.4 Evaluation Strategy

In section 3.3, we mentioned adapting three example games that were previously created/modified to use GitLab as a UI to perform the same behaviour, but instead using method calls to our Unity package instead of coding everything from scratch. We plan to analyse these game implementations to evaluate the effects of our package.

This section discusses our main goal, how, and what we plan to do to achieve it. The next section looks at the work we have done to prove the feasibility of this project.

4 Feasibility

In this section, we go over the work done to prove the feasibility of the project before we start work on it. In section 4.1, we look at the coding environment we have chosen for our prototype. In section 4.2, we look at a diagram of the functionality planned for the project. In section 4.3, we discuss the actual prototypes built and anything we did to gain further understanding of our tools. In section 4.4, we discuss the experiment we plan to run to gain our main design for our package.

4.1 Coding Environment

For our coding environment, we have opted to use C# and the Unity Game Engine due to our having prior experience with Unity. Additionally, Unity's HTTP request documentation is more in-depth than other game engines like Unreal Engine's HTTP request documentation, which, although it can send HTTP requests, its documentation for such is much smaller and less in-depth than Unity's. Additionally, both engines have learning websites to help teach concepts, these being: Unity Learn and Unreal Academy. Due to the difficulty of setting up Unity with GitHub/GitLab, we have opted to use the Unity Version Control for our versioning as it is much easier to set up and built in. We have also opted to manually upload important versions of our files to GitLab to better safeguard against loss of files. For our GitLab host, we have opted to use the Heriot-Watt-hosted student GitLab website to host our repository, where we can test the effects of our package. We will also be using the main GitLab website to host another repository for testing the package's capability to work for other GitLab hosts.

4.2 Design

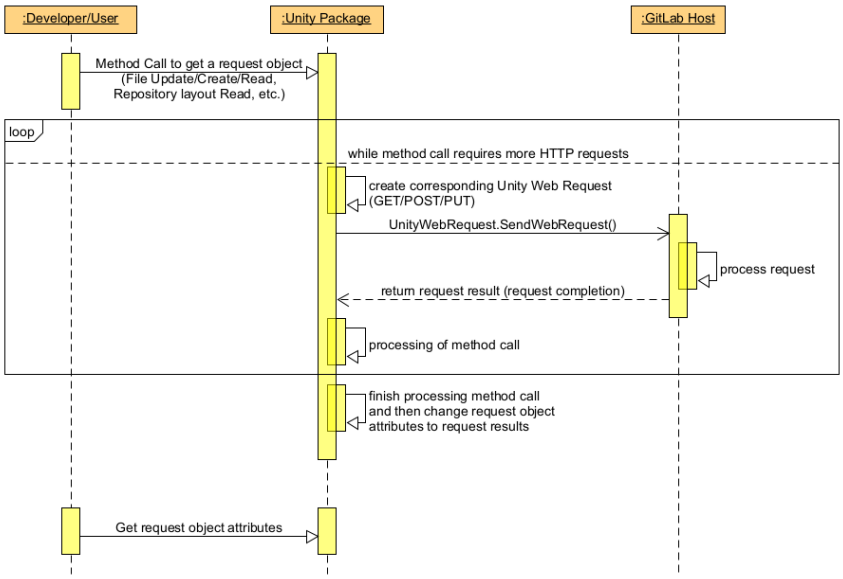


Figure 1: A sequence diagram depicting the overall communication between the user, our package and the GitLab host.

Depicted in Figure 1 is the structure of how our package works. Users will use method calls and receive a request object back, then the package will send HTTP requests as needed to GitLab hosts to complete the goal of the method. After the method is complete, the results are placed in the request object. The user can read the request object attributes at any time.

4.3 Prototypes

For our prototype, we modified some code we created to make HTTP requests to a specific Heriot-Watt student GitLab repository to create a set of abstracted method calls that could be called to achieve the same functionality – abstracting the code from the scripts to single method calls. These method calls cover creating a file, updating the contents of a file, reading the contents of a file, and continuously checking for the existence of a file. We then created another script calling the functions from the abstraction method calls script (although waiting for a response was to be handled by the method calling script). Doing this allows us to better understand how to use the Unity engine, how to use the Unity HTTP request methods, and how to make requests using the GitLab API. It also allows us to gain an initial idea of what our method calls could look like, gain experience with using C# and Unity, and gain an idea of how our method calls would need to be used. Through experimentation during development, we decided to use the NewtonSoft package for our deserialization and serialization.

4.4 Experiment/Focus Groups

As mentioned in section 3.3, we are designing most of our Unity package's method calls based on our experiment results. This experiment entails having participants create abstract pseudocode method calls for methods that would achieve the same functionality as code snippets from our example games. We ask participants to note down extra thoughts or reconsiderations (and the reasoning for them) they have throughout the experiment, so that we can see their entire thought process, as this may be connected to the usability of the package. We also ask users to answer Likert scale questions concerning computing capability so that we can detect patterns between skill and answer. We have created prototype copies of the consent form, participant information sheet, and experiment questionnaire sheet, which can be found in the appendices.

This section discusses the work done to prove the feasibility of our project, including a prototype. The next section concludes and summarises the main points of the project.

5 Conclusion

In section 5.1, we discuss our motivations for our goal of developing a GitLab UI game development package and what impact this goal would have. In section 5.2, we discuss what we need to do to accomplish the goal we discussed throughout the paper.

5.1 Motivation and Goals

A wide variety of computer science serious games exist, and quite a few incorporate coding into their gameplay. To facilitate the further creation of these games, we want to create a Unity package that facilitates the development of games that use a GitLab UI. This is so that not only do students gain better experience with professional platforms for versioning, but also so these types of games propagate further and reach more students, meaning these games have a stronger impact.

5.2 Proposed Work

To accomplish the goal written in section 5.1, we propose that we shall: create three example games without abstraction, run an experiment to discover the best method calls for our package, implement our package in Unity, re-implement the three example games using our package, and analyse the effect of our package on the games' code.

6 Implementations and Contributions

This section discusses the work we have done in creating three examples of games that use a GitLab repository/repositories as a user interface, as well as the implementation of our final GitLab game UI package for Unity.

6.1 Games Concepts

In this section, we created three games to show the different applications and capabilities of using a GitLab repository as a user interface, as well as the ability to meet the requirements we set out in section 3.2. This will also allow us to demonstrate the package's ability to meet our requirements later on when we adapt the three games to use our package instead of implementing the web requests manually.

Each game will require players to submit a repository's host name, project number, and project access token (optional for game 1 if it is a public repository), which will be used to validate the game's capability to interact with the games before the game can start (for example, if a game needs to be able to create and delete files then creation and deletion will be tested). These details will designate that repository to be a "play area", which the player will use to interact with the game.

Game 1:
Our first game centres around a character named Bob. The player is to submit a repository that represents a house through the use of file and directory names (for example, a directory named "bedroom" could contain a file named "toilet" and therefore the bedroom would contain a toilet). When submitting the repository details, the player is also asked for a personal access token.

Bob presents a set of 5 rules to the player for the house, which are:

1) At least one bedroom must contain a toilet
2) Stoves must be inside a kitchen
3) Closets must not contain sinks
4) Bob must be inside a bedroom
5) The house must contain more than one bedroom

The game considers a file/directory to be an object if it contains that word in the name. For example, a file/directory named "bedroom1", "Bob'sBedroom", or "NotABedroom" would all be considered bedrooms. For rules like rule 3, as it does not specify that Bob wants a stove in his house, the rule is still considered met if there are no stoves in the house because there is no stove outside of a kitchen. If the player submits the repository and the contents of the repository successfully pass the checks for all 5 rules, then the player wins. The game will use the player's personal access token to address the player by name, and it will also inform the player of the last time of update of the entire repository and of the first file found meeting rule 4's condition named "Bob" (this will return an error message if the repository's "Bob" file is actually a directory).

Game 2:
Our second game centres around making changes to a repository. The game will create a target file in a random location within the repository (which means it can be within any directory) with the name "targetx" where x is the lowest number that causes the file name to be completely unique in the entire repository (which means if the repository already

has files/directories named “target1”, “target”, and “target4” then the file created will be named “target2”).

Once the file has been created, the game will tell the player to change the contents of the target file. The player is given three lives and will lose lives if they:

1) Change the contents of any file that is not the target one
2) Delete any file
3) Create any files

Deleting a repository causes the player to lose a life for every file inside the repository. Upon losing all lives, the player loses. If the player changes the contents of the target file, they win.

Game 3:

Our third game adapts a pre-existing game to prove the capability of adapting real games and to prevent bias in our results from all the games being designed for the purpose of using a GitLab UI.

The game chosen has a grid of nodes that each point in one direction (up, down, left, or right). Nodes can be neutral, on player 1’s team, or on player 2’s team. A node on either player 1 or player 2’s team will cause the node directly in front of the direction it is pointing in to join its team. The game is turn-based, and each player chooses a node to rotate before moving on to the other player’s turn. When a node is chosen to be rotated, its direction is turned 90 degrees to the right. A node is in a “cluster” with all the nodes affected by its direction and all the nodes whose directions affect it (aka. the tree of nodes leading to the nodes that point to it and the tree of nodes leading from the node it points to). At the very beginning of the game, both teams start with 1 node: player 1’s node starts at the very top left corner, and player 2’s node starts at the very bottom right corner. The player to first remove all instances of the other player’s nodes from the grid wins. The player can play against an AI of different difficulties or against another player.

We have adapted this game to create a file named “Board” within the given repository’s initial area (meaning it does not go into any directories) or, if it already exists in the repository’s initial area, to reset the contents of the “Board” file to the default board state. The “Board” file contains a grid of x’s corresponding to the nodes in the grid within the actual game. When the board is updated, the game detects this and acts accordingly:

- If the board has one character changed and it corresponds to one of the player’s nodes, then the corresponding node is rotated.
- If the board has more than one character changed or the chosen node is not one of the current player’s nodes, then nothing happens in the game.
- The board is reset to its original state once any changes are detected.

The player can still choose to play the game normally through clicking nodes – showing the capability of a GitLab UI to work in tandem with other user interfaces. Once the game has ended, an issue is created in the repository stating the outcome of the game.

If the player chooses to play against another player instead of an AI, then the game works the same, except two repositories are accepted, and each repository only allows moves for its respective player. Issues are created in both repositories at the end of the game that state the outcome.

6.2 Experiment

Once the initial versions of the three games were created, experiments were performed to determine the style of the package that potential users would prefer. This would help decide parameter types, granularity, method call names, etc. This is because we want to ensure that the parameters and method calls of the package are as intuitive to use as possible, whilst still keeping the package capable of performing the tasks our users require of the package.

The experiment contained 4 files: The questionnaire file, the input parameter file, the task 3 file, and the task 6 file.

The task 3 and task 6 files contained a copy of all of the relevant scripts from the three games described in section 6.1, but the task 3 file also had small descriptions of the games and other small notes, whilst the task 6 file did not. The code found in both files was thoroughly commented to ensure participants could easily understand the code.

The input parameter file contained screenshots of the GitLab REST API input parameter tables for each type of web request I used in the three games' code, and also the "Create a commit" web request, which, whilst not used in the games' code, could be used to perform multiple of the web requests found inside the games' code simultaneously.

The questionnaire file contained all of the instructions for what the user was meant to do for tasks 1 to 6. The tasks are as follows:

Task 1: The participant was asked for their programme and year.

Task 2: The participant was asked to answer 3 Likert-scale questions regarding their experience in Games programming, API programming, and DevOps. They were additionally asked to comment any experiences relating to those three areas that they wanted to share.

Task 3: The participant was asked to read and familiarise themselves with the code found in the task 3 file.

Task 4: The participant was asked to choose a formatting style and to write down any changes in their thought process/answers when answering the questions that followed.

Task 5: The participant was asked to make a list of method calls that would abstract the code they read in task 4 (the three games' code). They were asked to centre the usage of the method calls towards the development of GitHub/GitLab UI games. They were asked to provide for each method call: explanations of the parameters, descriptions of the method's behaviour or overall purpose, and any extra functionality or extra information they wanted to mention.

Task 6: The participant was asked to recreate the three games' code by modifying the copies of the three games' code found in the task 6 file to use their method calls created in task 5. They were told that their modifications did not have to be functional and only required them to use pseudocode.

The purpose of tasks 1 and 2 is to allow us to detect if there are patterns of preferences amongst participants with similar levels of experience, whether in computer science as a whole or in more specific areas such as API programming. This is so we would know to try to balance accommodating these different audiences if such patterns made themselves known in the results.

The purpose of task 3 is to prepare the participant for making the list of method calls in task 5, as simply asking them to abstract the code without first having time to read it might have overwhelmed some participants.

The purpose of task 4 is to give the participants freedom in how they wish to format their answers, whilst also giving us the opportunity to ask them to note down any changes in their thoughts or answers.

The purpose of task 5 is to allow us to see the main style preferences of the participants, such as what kind of data types and names they prefer for the parameters, what naming conventions make more sense for the methods, and what sort of methods they would prefer to have at their disposal.

The purpose of task 6 is to make participants think a little more in-depth about how they want their methods to be used, how they would work, whether or not their method calls cover all the functionality they would want as a Unity GitHub/GitLab UI game developer, etc. This leads back to task 4, where they are allowed to make changes since participants may want to go back and change or add more methods to their method list in task 5.

Overall, in this section, we have described the structure, contents, and layout of the experiment whilst also explaining the purpose of each part of the experiment.

In the next section, we will analyse the results of the experiment to extract a suitable format of method calls for our package and design an initial version of the package.

6.3 Results Analysis

In this section, we analyse the results generated by conducting the experiment described in section 7.1 to design the method calls for our GitLab UI game development package.

A total of 6 participants were recruited, but participant 3 did not show up. Participants will be referred to by anonymous identifiers – these being “Px” where x is their participant number (e.g., P6 or P1). As participant 3 did not show up, no information can be extracted from their results.

When analysing the results, participants’ answers to the questionnaire sheets were compared, with the most attention being paid to task 5, whilst tasks 1 and 2 were used to detect any experience-related patterns amongst the participants’ answers. The Task 6 files were converted from Word files into plain text files and compared to the original version of the task 6 file using Meld to more easily detect where and what changes were made by participants.

Imitation of the original GitLab REST API

Multiple participants had method calls in their answers that simply abstracted the relevant parts of the original GitLab REST API, with P1 adding back the original inputs from the input parameter sheet.

P4 had one method that would take care of a majority of the web requests, e.g., GET file or UPDATE file. But, because all other participants separated them, our package will also keep these web requests separate.

GitLab API request	Participant 1 method name	Participant 6 method name
Delete a file	DeleteRequest	DeleteFile
Create a file	CreateRequest	CreateFile
Get a file's contents	GetFile	GetFileContentsFromPath
Get a single user	GetSingleUserRequest	GetUser
Retrieve a test report for a pipeline	GetPipelineRequest	GetPipelineReport
Get Repository Tree Layout Request	GetRepositoryTreeRequest	GetRepoTree
Get a repository's list of commits	GetRepositoryCommitsRequest	FetchCommits
Get a specific commit's details	GetCommitRequest	FetchCommitDetails

Table 1: A table of methods from participant 1 and 6 that are equivalent to an GitLab REST API request.

The list of most commonly taken GitLab REST API request functionalities for participants' method calls can be seen in Table 1, where we can see that only P1 and P6 seemed to copy each web request individually. Although P6's GetPipelineReport can also return the pipeline object, since this behaviour is not seen in other participants' answers, our adaption of this will leave this functionality out. The methods seen in the table are the most common amongst the participants and therefore will be featured within our package. Since not all of the functionality of P6's "GetPipelineReport" method (which can return a pipeline object or a pipeline report depending on the value of the "report_type" parameter) is found in the GitLab API request, that extra functionality will not be included in our package.

In terms of parameters for each web request, P1 takes in all the parameters available from the original GitLab REST API, whilst P6 only takes in the repository branch, ref (meaning which commit, branch, or tag to apply the web request to), report type to ask for a pipeline instead of a pipeline report, datetime, and API version. API version is unapplicable as other versions of the API were not available for use. Report type is also unapplicable, as it was used to enable extra functionality that we have already decided that we are not including in our package (the ability to retrieve pipeline objects). For ease of use, we shall follow P6's example and add datetime, branch and ref to the relevant requests in our package. We shall take in two datetime parameters, which will be used to indicate the start and end of what commits the user wants returned. For essential functionality, we shall also take the commit message parameter from P1.

Default parameters

P6 included the ability for their method calls to have default values for parameters they thought developers would want to omit specifying. P1 did something similar with nullable parameters. As it is more flexible, we have opted to use default parameters.

Specialised methods

Participants who reported a lower level of experience in task 2 created methods with niche applications. This can be seen with P2 (who reported a neutral response to having experience in API programming), creating two methods focused on checking file existence in repositories, which can be seen in Figure 2 and Figure 3. P4, (who disagreed with having experience in Dev Ops), although having the most broadly applicable web request method, created two of the most specific methods, which can be found in Figure 4 and Figure 6.

In contrast, participants who only reported having a normal or higher than normal amount of experience in task 2 wrote more broadly applicable methods. For P1 (who agreed to having experience in all three areas in task 2 and added an extra comment stating extra experience in games and API programming), their answers closely mirrored the GitLab REST API given to them, which has a broad applicability due to needing to cater to any users of the REST API. P6 (who agreed to having experience in all areas except for API programming, where they strongly agreed) also has most of their methods closely follow the GitLab REST API.

In conclusion, to cater to both experienced and inexperienced users of our package, we should attempt to have methods with small and large ranges of use cases that would therefore abstract small and large amounts of work. The default parameters discussed in the last section would be useful here, as inexperienced users could ignore the extra parameters.

GitLab Host URL, Project ID, and token parameters

P2, P4, and P6 all chose to have each method call they created take in the GitLab Host URL, project ID, and token parameters (which we shall call the repository details) each time they are called whilst P1 opted to have a method call creating a separate object for each project so that the repository details would not have to be passed in each time. It should be noted that P1 actually passed in the repository details as parameters originally, but changed it to use the object creation technique later on. Since most of the participants kept this design and most at least started with it, we must conclude that passing in the repository details as parameters is the intuitive way for our package to take in these details.

Check for a file inside the repository method and Check for a file at a specific location method

```
bool checkForFile (string mainUrl, string projectcode, string fileName)

MainUrl is the gitlab host, projectcode is the id of the users repository and filename is the name
of the file we are checking exists in the repository

Could be used to check if a certain file exists in the repository.
```

Figure 2: Participant 2's method call that checks for a file inside the given repository

As seen in Figure 2 - P2 has created a useful method call that lets the user know if a certain file exists in the repository. This method abstracts the process of having to use a get repository tree request and manually looking through the returned objects – for this reason, we have decided to add this to our package.

Bool CheckFileAtLocation(string mainUrl, string projectcode, string filePath, string name)

MainUrl is the gitlab host, projectcode is the id of the users repository, filepath is the repository filepath to the file and filename is the name.

Could be used to check that a file is in the correct location in a repository

Figure 3: Participant 2's method call that checks if a file with a given name is at a given path in the given repository

P2's method to check for a file with a specific name at a given file location in the given repository (as seen in Figure 3) would be a simple process as it only requires using a get file request and correctly checking the returned status code, and it would abstract the need to understand the returned status codes away from users. Therefore, we are going to add this method to our package. For the filePath and name parameters, we shall be merging them into one filePath parameter so it matches the style of the more common create a file method, and so that users don't have to extract file names from file paths.

FUNCTION validateInputs(TMP_InputField URL, TMP_InputField ProjectNum, TMP_InputField Auth){

```
      RETURN ((Url.text is string) && (ProjectNum.text is string) &&
(Auth.text is string) && (Int32.TryParse(ProjectNum.text, out int
NumberValue)))
```

}

The URL, ProjectNum, and Auth parameters are the inputfield UI elements that will be present in the scene and will be where text is entered.

This function takes the 3 input fields and validates them, its main function is to simplify the check case whenever a page has them.

Figure 4: Participant 4's method call that checks the given repository details are of the right types

Repository values validation method

In Figure 4, we can see a method call by P4 that checks the validity of repository details passed in (but not the access capability). Further improvements to this method could be changing the parameters passed in to string datatypes, as strings are the data type extracted from these input field objects in the games from the ".text" variables and would make the method call much easier to use and applicable to more use cases. Additionally, the method call could be changed to check the length of the parameters to make it easier for users of our package to comply with OWASP security guidelines although these checks would be made in the other methods as well. Our package shall implement this method with the discussed changes made.

File path methods

```
FUNCTION elementOverlap(TreeObj repoNode, STRING elem1, STRING elem2){

      RETURN repoNode.Contains(elem1) && !Array.Exists(repoNode, element =>
element.Contains(elem2));

}

This function simplifies the checks for Game 1 where it needs to be checked
for overlapping elements such as Bob and Bedroom, Stove and Kitchen, Closet
and Sink, Bedroom and Toilet, and allows for more checks.

The repoNode is the node in the TreeObj structure that has the saved
information for that repo at that time, with folders and files. Elem1 and
Elem2 are the 2 strings you are checking if they appear together at any point
in the node.
```

Figure 6: Participant 4's method call that returns whether or not two given strings are both in the same repository structure

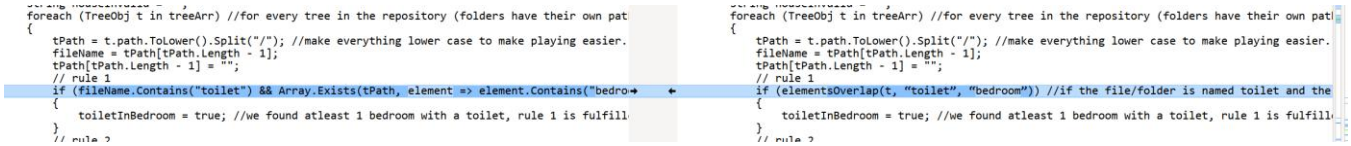


Figure 5: A comparison of the original game 1 code, that is checking if a toilet file/directory exists anywhere inside a bedroom directory, with participant 4's version of the same code (Screenshot of a Meld comparison. Part of participant 4's answer to task 6).

In Figure 6, P4 has created a method call that checks if two strings are in the same file path. From Figure 5, we can determine that this was intended to replace the part of game 1 which checks that parameter elem1 is the current file path's file name and that the parameter elem2 exists somewhere in the file path. This contrasts with the pseudocode given, which checks that repoNode contains both elements instead, although we can see the attempt to recreate the original game code due to the similarities between them. To make this method functional, we adjust the method to check that elem1 is the given file path (repoNode)'s file name and that elem2 exists somewhere in the file path.

However, this functionality is overspecialised for a general GitLab UI package. To make the method applicable to more use cases, we can split this functionality up into several methods: one method to take in a treeObj or file path and return the file/directory name it points to and one method to check for a string in the file path (with default Boolean parameters as flags to say if the string should be a standalone directory name, if it can make up part of a directory name, and if the check should be case-sensitive).

Method Call Naming Conventions

A majority of all the method calls created across all participants started with a verb with the noun being affected coming second in the name (Extra words are added if it helps with the understanding of the method's purpose). This can be seen in methods such as "CreateRequest" or "GetSingleUserRequest" from P1, "checkForFile" or "CheckFileAtLocation" from P2, "validateInputs" from P4, or "FetchCommitDetails" from P6. It should also be noted that every participant capitalised each word except for the starting word in the method named, which was equally capitalised and uncapitalised amongst the participants. Following (Unity, no date)'s recommendation for using Pascal Case for not-local method names, we have decided to capitalise the first letter in each word of our method calls, including the first letter of the first word.

Parameter Naming Conventions

Almost all the participants chose to keep their parameter names entirely lower case or used camel case (with most switching between the two in the same method calls). Due to this, the easier readability of camel case compared to the parameter names being

completely lower case, and (Unity, no date)'s recommendation to use camel case for method parameters, our package shall use camel case for its method parameters.

In terms of the actual parameter names, the Host URL is referred to as "githubURL" by P6, "url" by P4, "mainUrl" by P2, and "repositoryName" by P1. Since a majority of these use "url", we shall use "hostUrl" as that is the most concise yet still accurate name from this trend. Through a similar process, we have extracted "projectCode" and "token" as parameter names for the project code and project access token.

Parameter Ordering Conventions

Most participants started their parameters with the GitLab Host URL first, the project code second, and the project access token third. For example P2 had "string mainUrl, string projectcode, string filePath" at the start of both of their method calls, P6 had "githubUrl: str, projectCode:str, token: str" at the start of a majority of their method calls' parameters, P1 had "String repositoryName, String id, String? Token" at the start of their method call that created a project's web request object, and P2 had "ENUM reqType, STRING url, JSON formData, STRING token" at the start of the multi-use web request object's parameters. We can therefore conclude that we should always start with these three in that order.

GetFileContentsFromPath(githubUrl: str, projectCode: str, token: str, path: str, ref: str = "HEAD")
This method accepts parameters: github URL, project code, api token and path of the repo tree and returns the file contents from the repository tree. This method might be helpul if we want to compare two files contents inside separate repositories. It also includes the ref parameter so that we can later use it to fetch file contents of two different commits or branches. Renamed from GetFileContentsFromTree -> GetFileContentsFromPath for better method name convention

Figure 8: one of participant 6's method calls

HTTPStatusCode UpdateFileRequest(String branch, String commitMessage, String content, String filepath, String? authorEmail, String? authorName, String? encoding, Bool? executeFilemode, String? lastCommitId, String? startBranch)

Updates a file at target filepath

myRepo.UpdateFileRequest("main", "Updated file", "filecontents...", "/...")

GetRepoTreeReturn GetRepositoryTreeRequest(String? pageToken, String? pagination, String? path, Int? perPage, Bool? recursive, String? ref)

Gets the repository tree

GetRepoTreeReturn tree = myRepo.GetRepositoryTreeRequest()

Figure 7: Two examples of participant 1's method calls

However, we still need to determine how to order the rest of the parameters. As you can see in Figure 7, for P1, a majority of their methods order their parameters by putting all the non-nullable ones before all the nullable ones. Similarly, for P6, default parameters always come after all of the non-default parameters. Additionally, for methods sharing parameters, P6 will put those shared parameters together within both methods and put them in the same order – this can be seen in their methods CreateFile with the parameter list: "githubUrl, projectCode, token, path, commit_message, branch, content" and DeleteFile with the parameter list: "githubUrl, projectCode, token, path, commit_message, branch" where we can see the shared parameters path, commit_message, and branch are placed together immediately after the initial three repository details parameters, that they are kept together, and kept in the same order. From this we can conclude that we should have the repository URL as the first parameter, the project code second, the project access token third, and that nullable parameters must come after all non-nullable parameters, and any methods that share parameters other than those main three should keep those parameters together, in a similar position within the method's parameter list, and ordered in the same way.

Additionally, using our own judgement, we can conclude that we should order parameters in terms of importance to the method call. For example, in a method call that will delete a file, the parameter for the commit message should come after the parameter for the file path because the file path is a more important variable for knowing which file to delete.

Method return parameters and Abstraction of success codes

Two participants expressed a desire for the success codes to be abstracted for them. As seen in Figure 9, P1's method calls returned success codes as an enumeration rather than the actual success code itself, with the style of pseudocode showing the success and error returns causing different code to be run.

```
Match GetSingleUserRequest () {
        Success(user) => {
                profile = user;
                GreetUser.text = "Dear " + profile.name + ",";
        //write their name in a greeting on the win/lose page.
                personalTokenValid = true
        }
        Error(errorcode) => personalTokenValid = false
}
```

Figure 9: one of participant 1's pseudocode method calls in their task 6 code

Changing the method to also return a true/false on finish to signal the caller whether it succeeded or not without having to check the result.

```
FUNCTION webRequest(ENUM reqType, STRING url, JSON formData, STRING token, INT
successCode, OUT result){
```

Figure 10: One of participant 4's changes to their multi-use method call

For P4, one of the changes noted by them was "Changing the method to also return a true/false on finish to signal the caller whether it succeeded or not without having to check the result." which lets us know that whilst this was a later change and could therefore be a less intuitive decision, it should be noted that it was an important enough feature to go back and change the previous design.

As can be seen in Figure 10, P4 changes their multi-use method call to return true for a success and false for failure of the web request that is sent. Like before, this was a later change and so should be noted to be unintuitive but important enough to change. Therefore, we can conclude that abstracting the success codes is important.

Additionally, the success and error objects that run additional code in P1's code (as seen in Figure 9) can be easily recreated by returning a Boolean along with the result in a separate parameter, much like in Figure 10. Therefore, we can conclude that we should return the abstracted success code whilst still returning the relevant information. For our initial package design, we have opted to return a Boolean and the information separately rather than in one object, as this would make the return parameters easier to work with rather than having users have to adapt to a custom data type. We can do this by taking inspiration from P4's answer, where they use out parameters to separately return another variable. To preserve the chosen return style, we shall have to keep the internal functionality of the method calls synchronous. Whilst we previously agreed to have repository details as the first parameters, we shall be making an exception for out parameters in order to group them together with the method's return value visually in the method's definition and method call.

For our out parameters, we still have to deserialize the data returned to us from the web requests we abstract. In the experiment, a large majority of the participants chose to use the deserialization class objects we created instead of making their own smaller classes that would have fewer variables and would therefore be easier to use. To follow this trend, we shall keep our variables as the return object's type and ensure that users of the package

have access to a large variety of variables, giving the package more freedom in the way it is used.

6.4 Initial Package Design

In this section, we take the information and design decisions made in section 7.2 and implement them into an initial pseudocode design of our package, consisting of only the method calls.

We will also make certain decisions for the package based on our requirements from section 3.2. For the Web Requests, Preservation, and Transparency sections of the requirements, any functionality not covered by the methods we gathered will be added in as extra method calls, which should follow the method naming conventions, parameter naming conventions, parameter ordering, as well as any relevant rules we determined in section 7.2. For the Usability section of the requirements, any of these requirements not already featured in the method call structures will be added to all of them where possible. For the Security requirements, all methods will be made to adhere to them if they do not already. Separate from the method checking that the repository details given are of the right type and length, each method should internally check this as well to ensure proper security standards are easily met by users of our package. The Documentation requirements are not relevant to this section.

When creating this initial design (mostly during the game creation phase of the original three games), I referenced NGitLab's files to see what data types they were making the variables, which I also needed for my deserialization class objects (which the web requests would deserialize the responses into).

Notation: variableType var=value gives value as the default value for the parameter if it isn't passed in. out variableType denotes an out variable. Notes about individual methods are written under the method call in italics. Custom objects are named Obj (For example, UserObj for deserialized information about a user).

The ordering of parameters is generalised to this: out response, out contents, hostUrl, projectCode, token, filePath, Id, originalLayout, content, commitMessage, branch, ref, and encodeContents. (Less common parameters are added on a case-by-case basis in such a position that they don't disrupt the pre-established pattern or so that they feel more intuitive (so out parameters first, repository details second, upload content third, etc.).

Boolean DeleteFile(string hostUrl, string projectCode, string token, string filePath, string commitMessage="", string branch="main")
Boolean CreateFile(string hostUrl, string projectCode, string token, string filePath, string content, string commitMessage="", string branch="main", boolean encodeContents=false)
Boolean GetFile(out FileObj contents, string hostUrl, string projectCode, string token, string filePath, string ref="HEAD")
Boolean GetUser(out UserObj contents, string hostUrl, string personalAccessToken) *"personalAccessToken" is used here to explicitly differentiate it from the normal project access token parameter.*
Boolean GetPipelineUnitTestReport(TestReportObj contents, string hostUrl, string projectCode, string token, string pipelineId) *This method name is longer than the participants' but more self-explanatory.*

Boolean GetRepositoryTrees(out RepositoryTreeObj[] contents, string hostUrl, string projectCode, string token, boolean recursive=false, string ref=”main”)
Boolean GetRepositoryCommits(out RepositoryCommitListObj[] contents, string hostUrl, string projectCode, string token, string ref=null, string startDate = null, string endDate = null) *Ref here defaults to null rather than “main” because it isn’t a required parameter.*
Boolean GetCommit(out CommitObj contents, string hostUrl, string projectCode, string token, string commitId)
Boolean CheckForFile(string hostUrl, string projectCode, string token, string filename, string ref=”HEAD”, boolean excludeDirectories = false)
Boolean CheckForFileAtLocation(string hostUrl, string projectCode, string token, string filePath, string ref=”HEAD”, Boolean excludeDirectories = false)
Boolean ValidateRepositoryDetails(string hostUrl, string projectCode, string token) *Checks if the repository details given are of valid data types and length.*
Boolean CheckElementInPath(RepositoryTreeObj/string filePath, string element, boolean standaloneElement = false, boolean caseSenstivive =false) *filePath can either be of RepositoryTreeObj type or string type.*
String GetFileName(RepositoryTreeObj/string filePath) *filePath can either be of RepositoryTreeObj type or string type.*
Extra methods generated from requirements:
Boolean UpdateFile(string hostUrl, string projectCode, string token, string filePath, string content, string commitMessage = ””, string branch = ”main”, boolean encodeContents = false)
Boolean SaveRepository(out RepositoryStateObj contents, string hostUrl, string projectCode, string token, string ref = “HEAD”) *Returns a “layout” (custom repository object) which contains all the file paths and file contents.*
Boolean CheckChanges(out List<string> filesAdded, out List<string> filesDeleted, out List<string> filesChanged, string hostUrl, string projectCode, string token, RepositoryStateObj originalLayout, ref = “HEAD”) *Returns a list of any changes compared to the given layout.*
Boolean CheckFileChange(string filePath, Boolean? extraEncoding) *Returns true if the file at filePath has changed (so if it was deleted or contents changed)*
AutonomousDeleteExtra createAutonomousDelete(string hostUrl, string projectCode, string token, RepositoryStateObj originalLayout, string commitMessage = “deleting added file”, string branch = “main”, string ref = “HEAD”) *Starts a continuously polling method that deletes any extra files or directories created (compared to the original layout given).*
AutonomousReplaceMissing createAutonomousReplace(string hostUrl, string projectCode, string token, RepositoryStateObj originalLayout, string replaceCommitMessage = “replacing missing file”, updateCommitMessage = “updating file contents”, string branch = “main”, string ref = “HEAD”, Boolean encodeContents = false) *Starts a continuously polling method that replaces any missing files and changes file contents back to their original contents (compared to the original layout given).*
Void Stop() *Stops the asynchronous method the AutonomousReplaceMissing/AutonomousDeleteExtra object this was called on.*
String CheckAnyAutonomous()

Returns a string showcasing the repository details of any currently running AutonomousReplaceMissing or AutonomousDeleteExtra objects
RequestQueue GetRequestQueue() *Returns the request queue of the package instance. This allows users to search through and determine what jobs are running through the RequestQueue's RequestIDs' int id and string identity values.*
Boolean CreateIssue(string hostUrl, string projectCode, string token, string title, string description="")

Table 2: A table holding all the method calls in the initial design for the package

In the next section we look at our package after it has actually been implemented.

6.5 Final Package

In this section, we write out the finalised package design and discuss there differences from the initial package design.

The final package was created following Unity's guidance to create a local package. Its method calls can be found within Table 3.

The method calls were also reordered so that types of method calls would be grouped together. For example, CheckElementInPath and GetFileName are together due to mainly being helper methods, whilst GetFile and CreateFile are together as they are both abstracting singular GitLab API web requests.

Notation: type a/type b means there is an overload where the same parameter takes in a variable of type b instead of type a.

ID	Method Call
1	public bool CheckElementInPath(RepositoryTreeObj/string fileObj, string element, bool standaloneElement=false, bool caseSensitive = false)
2	public string GetFileName(RepositoryTreeObj/string fileObj)
3	public bool CheckForFileAtLocation(string hostUrl, string projectCode, string token, string filePath, string repoRef = "HEAD", bool excludeDirectories = false, ProjectCodeType projectCodeType = ProjectCodeType.Integer)
4	public bool CheckForFile(string hostUrl, string projectCode, string token, string fileName, string repoRef = "HEAD", bool excludeDirectories = false, ProjectCodeType projectCodeType = ProjectCodeType.Integer)
5	public Response CheckFileChanged(out string state, string hostUrl, string projectCode, string token, string filePath, string originalfileContent, string repoRef = "HEAD", ProjectCodeType projectCodeType = ProjectCodeType.Integer)
6	public Response CheckChanges(out List<string> filesAdded, out List<string> filesDeleted, out List<string> filesChanged, string hostUrl, string projectCode, string token, RepositoryStateObj originalLayout, string repoRef = "HEAD", ProjectCodeType projectCodeType = ProjectCodeType.Integer)
7	public Response SaveRepository(out RepositoryStateObj contents, string hostUrl, string projectCode, string token, string repoRef = "HEAD", ProjectCodeType projectCodeType = ProjectCodeType.Integer)
8	public Response GetUser(out UserObj contents, string hostUrl, string personalAccessToken)

9	public Response GetPipelineUnitTestReport(out TestReportObj contents, string hostUrl, string projectCode, string token, string/int pipelineId, ProjectCodeType projectCodeType = ProjectCodeType.Integer)
10	public Response GetCommit(out CommitObj contents, string hostUrl, string projectCode, string token, string commitId, ProjectCodeType projectCodeType = ProjectCodeType.Integer)
11	public Response GetRepositoryCommits(out RepositoryCommitListObj[] contents, string hostUrl, string projectCode, string token, string repoRef=null, string startDate = null, string endDate = null, ProjectCodeType projectCodeType = ProjectCodeType.Integer)
12	public Response UpdateFile(string hostUrl, string projectCode, string token, string filePath, string content, string commitMessage="updating file", string branch="main", bool encodeContents = false, ProjectCodeType projectCodeType = ProjectCodeType.Integer)
13	public Response GetRepositoryTrees(out RepositoryTreeObj[] contents, string hostUrl, string projectCode, string token, bool recursive = false, string repoRef="HEAD", ProjectCodeType projectCodeType = ProjectCodeType.Integer)
14	public Response CreateIssue(string hostUrl, string projectCode, string token, string title, string description="", ProjectCodeType projectCodeType = ProjectCodeType.Integer)
15	public Response DeleteFile(string hostUrl, string projectCode, string token, string filePath, string commitMessage="deleting file", string branch = "main", ProjectCodeType projectCodeType = ProjectCodeType.Integer)
16	public Response GetFile(out FileObj contents, string hostUrl, string projectCode, string token, string filePath, string repoRef = "HEAD", ProjectCodeType projectCodeType = ProjectCodeType.Integer)
17	public Response CreateFile(string hostUrl, string projectCode, string token, string filePath, string content, string commitMessage="create file", string branch="main", bool encodeContents = false, ProjectCodeType projectCodeType = ProjectCodeType.Integer
18	public bool ValidateRepositoryDetails(string hostUrl, string projectCode, string token = "", ProjectCodeType projectCodeType = ProjectCodeType.Integer)
19	public AutonomousDeleteExtra createAutonomousDelete(string hostUrl, string projectCode, string token, GitLabUI package, RepositoryStateObj originalLayout, string commitMessage = "deleting added file", string branch = "main", string repoRef = "HEAD", int secondsBetweenChecks = 3, ProjectCodeType projectCodeType = ProjectCodeType.Integer)
20	public AutonomousReplaceMissing createAutonomousReplace(string hostUrl, string projectCode, string token, GitLabUI package, RepositoryStateObj originalLayout, string replaceCommitMessage = "replacing missing file", string updateCommitMessage = "updating file contents", string branch = "main", string repoRef = "HEAD", int secondsBetweenChecks = 3, bool encodeContents = false, ProjectCodeType projectCodeType = ProjectCodeType.Integer)
21	public string CheckAnyAutonomous()
22	public void CancelAllAutonomous()
23	public RequestQueue GetRequestQueue()

Table 3: A table displaying the main methods for interaction with the final package that was created

Additional overloads are provided for each web request sending method that abstracts a singular GitLab API request (more specifically: method calls 8 to 17) adding a ResponseObj response out parameter so users can access the response code and response message (which can be custom for scenarios like invalid details or in the case of the web request failing will be the web request's download handler text). Additionally, the AutonomousDeleteExtra and AutonomousReplaceMissing classes have a Stop() method. which stops the instance it is called on from continuing its asynchronous functionality.

It should be noted that instead of Boolean, most of these requests return a Response object (which can have the values Success, Error, InvalidDetails, and Cancelled) – this was a decision made during package creation as it would allow users to know if their web request failed, got cancelled, or the details given were invalid instead of just returning false for each case. The enumeration return still meets the requirements we wanted the Boolean return to fulfil from our experiment analysis and is therefore an acceptable change. Additionally, the parameter ref has had its name changed to repoRef due to clashes with the ref keyword. Finally, the projectCodeType parameter at the end of most of the method calls takes in an enumeration with values "Integer", "Path", and "EncodedPath". By defining the project code type we can ensure we validate it as the correct type (so int or string) and also so that it can be encoded properly if need be.

For the autonomous web requests (AutonomousDeleteExtra and AutonomousReplaceMissing), their asynchronous functionality is started when they are created (using createAutonomousDelete or createAutonomousReplace) and stopped either by setting their working variable to false or removing them from the package's personal AutoDeleteCatalogue or AutoReplaceCatalogue, respectively. Calling Stop() on any AutonomousDeleteExtra or AutonomousReplaceMissing object's Stop() method removes all instances of that object from the catalogue and therefore also stops its asynchronous functionality. For the asynchronous method calling in the autonomous web requests, we attempted to use Coroutines (which needed to be on a script in the game scene to work and therefore would not be suitable for our package), as well as Tasks and Threading, which both said only the main thread could create. UniTask was used in the implementation of these autonomous web requests to allow us to call the method asynchronously away from the main thread, but still be able to create Unity Web Requests to send.

One issue with my final implementation is that there is a small time delay when using it compared to making the web requests manually. I believe this would be because I am doing a manual while loop waiting for web request sending permission to be available for a request to begin, which also includes a sleeping command inside it to try to reduce the effect of the busy wait.

In the next section, we aim to look at the effect of this package once we actually put these method calls into use within Unity games.

7 Evaluation

7.1 Analysis of Actual Package Usage

In this section, we aim to analyse the effect our package had when we used it to implement the three games instead of coding them line by line without the package.

The same games described in section 6.1, and which featured in the experiment, were recreated using the package. The package use was carefully used to try and imitate the way the game was implemented originally as closely as possible so as to avoid bias. Bracket spacing and indentation style were also copied to the best of our ability. Small pieces of code that are redundant can be found in some of the original versions of the code, but as these are fewer than 5 for all original versions, the effect has been deemed minimal, and the mistakes are kept as is to keep the analysis more genuine.

Game 2 also has two extra implementations. One which makes use of the asynchronous autonomous functionalities of the package instead of imitating the original's method calls and another which makes use of the CheckForChanges method (a method to take in a saved state of the repository and reports back three lists – one of files whose contents have changed, one of deleted files, and one of files added since the save state. For Game 1, we also made a small change for all versions that fixed a bug – this does not affect length, however, so it is irrelevant.

In Table 4, we can see the length of the scripts found in the 3 games for each implementation. "x"s are displayed in the table where there is no implementation. MenuButtonScript.cs was not included as it does not make use of web requests and therefore would not be affected. Within Table 4, from the original implementation to the normal package implementation, we can calculate a length decrease of 44.39% for Game 1's MainScript.cs, a 30.74% decrease for Game 2's MainScript.cs, a 30.47% decrease for Game 3's InputRepoButton.cs, and a 8.47% decrease for Game 3's GameController.cs. From the original implementation to the autonomous package implementation, we can calculate a length decrease of 48.01% for Game 2's MainScript.cs. If we compare the means of the original implementation lengths (468.75) with the means of the normal package's implementation lengths (334.25), then we get 134.5 lines decreased on average per script – a 28.69% decrease.

The main reasoning for such a decrease in length can be mainly attributed to two things:
1) Every web request was reduced to a method call.
2) All of the custom deserialization objects were moved into the package.

For the second point, the number of lines taken up by custom deserialization classes in the original scripts is: 13 for Game 3's GameController.cs, 22 for Game 3's InputRepobutton.cs, 167 for Game 1's MainScript.cs, and 23 for Game 2's MainScript.cs. By subtracting these values from the original implementation lengths, we can calculate a length decrease of 22.33% for Game 1's MainScript.cs, a 27.58% decrease for Game 2's MainScript.cs, a 23.19% decrease for Game 3's InputRepoButton.cs, and a 6.18% decrease for Game 3's GameController.cs. Between the mean of the original implementation lengths without the custom deserialization classes (412.5) and the mean of the normal package implementation is a decrease of 18.97%.

Script	**Original Implementation Length (lines)**	**Package Implementation Length (lines)**	**Autonomous Package Implementation Length (lines)**	**CheckForChanges method implementation length (lines)**
(Game 1) MainScript.cs	588	327	x	x
(Game 2) MainScript.cs	527	365	274	272
(Game 3) InputRepoButton.cs	229	159	x	x
(Game 3) GameController.cs	531	486	x	x

Table 4: Length table comparing different implementations of the scripts that make up the games

Therefore, we can say that on average, there is a 28.69% decrease in lines overall, but that the actual use of the web request methods themselves from the package causes a decrease of 18.97%. Although this may not seem like a large decrease, the level of understanding abstracted away from the user within this percentage is important, as most of these lines would have required the user to go and research the GitLab API and learn how to use Unity Web Requests.

7.2 Requirements Achieved

Table 5 summarises the requirements completed with a rationale as to why for the ones that weren't completed. Further details can be found in the appendices in a larger, more in-detail version of this table.

ID	Rationale	Priority
Requirements FR-1, FR-1.1, FR-1.2, FR-1.2.1, FR-1.2.2, FR-1.2.3, FR-1.3, FR-1.3.1, FR-1.3.2, FR-1.3.3, FR-1.4, FR-1.5, FR-1.6, FR-1.7, FR-2, FR-2.1, FR-2.1.1, FR-2.2, FR-2.3, FR-3, FR-3.1, FR-3.2, NFR-1, NFR-1.1, NFR-1.2, NFR-2, NFR-2.1, NFR-2.1.1, NFR-2.2, NFR-2.2.1, NFR-2.3, NFR-2.3.1, NFR-3, NFR-3.1, NFR-3.2, NFR-3.3, NFR-3.4, NFR-3.5, NFR-4, NFR-4.1, NFR-4.2, and NFR-4.3 are all complete.		
FR-2.1.2	Due to lack of time and the functionality of this requirement already being achieved by FR-2.1.1, this requirement was not implemented.	Could
FR-2.1.2.1	Due to lack of time and FR-2.1.2 not being implemented, this was also not implemented due to it being impossible without its parent requirement.	Could

NFR-3.6	This was ranked MoSCoW priority Won't previously and therefore we had said this would not be completed.	Won't
NFR-3.6.1	This was ranked MoSCoW priority Won't previously and therefore we had said this would not be completed.	Won't

Table 5: A table summarising the requirements completed and not completed, and if not completed, describes the reason for why this may be.

8 Discussion and Conclusions

The aim of this paper was to create a Unity package that would abstract the communication required for using GitLab as a UI. In section 6.1, we described three games that would cover a range of functionality.

Then, in section 6.2, the games were used in an experiment to discover what kind of method call conventions were intuitive for users. In sections 6.3 and 6.4, we analysed and created an initial package design, which was implemented and described in section 6.5.

In section 7, we begin evaluating our final package with section 7.1's script length analysis showing us a code decrease of 28.69% when using our package (with a 18.97% decrease when not considering custom deserialization classes). In section 7.2, we discovered that we have met all the main functionality required in the requirements we set out in section 3.2 (with the only requirements not met having their functionality duplicated by other requirements).

Therefore, due to us taking into account inputs from prospective users in sections 6.3 and 6.4, the range of functionality shown in section 7.2, and proven the practical application as well as measured effect on code length in section 7.1, we can safely state that our package is capable of decreasing the amount of code developers write during development of GitLab UI games in Unity, that it has a suitable range of functionality to support a range of games, and that it has a intuitive method call design.

Therefore, we conclude that we have successfully made a package that abstracts the creation of GitLab UI games in Unity and have achieved our initial goal of making it easier and more accessible to make coding games with a GitLab UI.

9 Further Work

Further improvements on the package would be to implement requirements FR-2.1.2 and FR-2.1.2.1, as these requirements would add a lot to the usability of our package. Users could get webhook signals instead of having to constantly check the repository and this would make things a lot more efficient for the games.

Another improvement would be to conduct further experiments with a larger pool of participants. These experiments could redo the experiment conducted in this paper with a larger pool of participants to observe larger trends in method call design, or they could conduct experiments on the package made in this paper to discover what parts of the method calls could be made more intuitive for users.

Also, making our package more efficient and removing things such as the busy wait whilst waiting for permission to send web requests is important. Along with this, we recommend investigating further into why our package's method calls are a bit slower than doing the web requests manually, as this lower speed could affect the end-users

playing experience as well as the testing experience of developers. Making it crucial to the end goal of the package.

Finally, A more specific and tested version of the deserialization output class objects could also bring more clarity into how to use the package, as well as bring more ideas in as to how to further expand the package's uses.

References

Bishop, J., Horspool, R.N., Xie, T., Tillmann, N. and De Halleux, J. (2015) 'Code Hunt: Experience with Coding Contests at Scale', *2015 IEEE/ACM 37th IEEE International Conference on Software Engineering*, Florence (Italy), 16-24 May. Institute of Electrical and Electronics Engineers. pp. 398–407. Available at: https://doi.org/10.1109/ICSE.2015.172.

Codecademy Team (no date) *What is Code coverage? Definition, Types & Best Practices*. Available at: https://www.codecademy.com/article/what-is-code-coverage-definition-types-best-practices (Accessed: 12 November 2025).

Dohmke, T. (2023) '100 million developers and counting', *The GitHub Blog*, 25 January. Available at: https://github.blog/news-insights/company-news/100-million-developers-and-counting/ (Accessed: 8 November 2025).

Fay, B. (2025) 'Computer science graduates struggle to secure their first jobs', *BBC News*, 22 August. Available at: https://www.bbc.co.uk/news/articles/cm21dvg8l1go (Accessed: 8 November 2025).

Feldmeier, P., Straubinger, P. and Fraser, G. (2023) 'PlayTest: A Gamified Test Generator for Games', *Gamify 2023: Proceedings of the 2nd International Workshop on Gamification in Software Development, Verification, and Validation*, San Francisco (United States of America), 4 December. Association for Computing Machinery (Gamify 2023), pp. 47–51. Available at: https://doi.org/10.1145/3617553.3617884.

Henning, M. (2009) 'API design matters', *Communications of the ACM*, 52(5), pp. 46–56. Available at: https://doi.org/10.1145/1506409.1506424.

Korkiakoski, M., Antila, A., Annamaa, J., Sheikhi, S., Alavesa, P. and Kostakos, P. (2023) 'Hack the Room: Exploring the potential of an augmented reality game for teaching cyber security', *AHs '23: Proceedings of the Augmented Humans International Conference 2023*. Glasgow (United Kingdom), 12-14 March, Association for Computing Machinery, pp. 349–353. Available at: https://doi.org/10.1145/3582700.3583955.

Laamarti, F., Eid, M. and El Saddik, A. (2014) 'An Overview of Serious Games', *International Journal of Computer Games Technology*, 2014(1), article 358152. Available at: https://doi.org/10.1155/2014/358152 (Accessed: 9 November 2025).

Luxton-Reilly, A., Simon, Albluwi, I., Becker, B.A., Giannakos, M., Kumar, A.N., Ott, L., Paterson, J., Scott, M.J., Sheard, J. and Szabo, C. (2018) 'Introductory programming: a systematic literature review', *ITiCSE 2018 Companion: Proceedings Companion of the 23rd Annual ACM Conference on Innovation and Technology in Computer Science Education*. University of Central Lancashire Cyprus (Cyprus), 30 June – 4 Jul. Association for Computing Machinery, pp. 55–106. Available at: https://doi.org/10.1145/3293881.3295779.

Maarek, M., Louchart, S., McGregor, L. and McMenemy, R. (2019) 'Co-created Design of a Serious Game Investigation into Developer-Centred Security', *Games and Learning Alliance: 7th International Conference, GALA 2018*, Palermo (Italy), 5-7 December. Cham: Springer International Publishing, pp. 221–231. Available at: https://doi.org/10.1007/978-3-030-11548-7_21.

McGregor, L., Chan, S.C., Wlodarczyk, S. and Maarek, M. (2022) 'Aligning a Serious Game, Secure Programming and CyBOK-Linked Learning Outcomes', *2022 IEEE European Symposium on Security and Privacy Workshops (EuroS&PW)*, Genoa (Italy), 6-10 June. Institute of Electrical and Electronics Engineers, pp. 486–495. Available at: https://doi.org/10.1109/EuroSPW55150.2022.00058.

Min, W., Mott, B., Park, K., Taylor. S., Akram, B., Wiebe, E., Boyer, K.E. and Lester, J. (2020) 'Promoting Computer Science Learning with Block-Based Programming and Narrative-Centered Gameplay', *2020 IEEE Conference on Games (CoG)*. *2020 IEEE Conference on Games (CoG)*, Osaka (Japan), 24-27 August. Institute of Electrical and Electronics Engineers, pp. 654–657. Available at: https://doi.org/10.1109/CoG47356.2020.9231881.

Myers, B.A. and Stylos, J. (2016) 'Improving API usability', *Communications of the ACM*, 59(6), pp. 62–69. Available at: https://doi.org/10.1145/2896587.

Ng, C.Y. and Hasan, M.K.B. (2025) 'Cybersecurity serious games development: A systematic review', *Computers and Security*, 150(C), article 104307. Available at: https://doi.org/10.1016/j.cose.2024.104307 (Accessed: 9 November 2025).

Open Worldwide Application Security Project (2025) *Application Security Verification Standard (ASVS)*. 5.0.0. OWASP Foundation. Available at: https://owasp.org/www-project-application-security-verification-standard/ (Accessed: 15 November 2025).

Rama, G.M. and Kak, A. (2015) 'Some structural measures of API usability', *Software: Practice and Experience*, 45(1), pp. 75–110. Available at: https://doi.org/10.1002/spe.2215.

Rojas, J.M. and Fraser, G. (2016) 'Code Defenders: A Mutation Testing Game', *2016 IEEE Ninth International Conference on Software Testing, Verification and Validation Workshops (ICSTW),* Chicago, Illinois (United States of America), 11-15 April. Institute of Electrical and Electronics Engineers, pp. 162–167. Available at: https://doi.org/10.1109/ICSTW.2016.43.

Straubinger, P. and Fraser, G. (2022) 'Gamekins: Gamifying Software Testing in Jenkins', *2022 IEEE/ACM 44th International Conference on Software Engineering: Companion Proceedings (ICSE-Companion)*, Pittsburgh, 22-24 May. Institute of Electrical and Electronics Engineers, pp. 85–89. Available at: https://doi.org/10.1145/3510454.3516862.

The British Computer Society (2025) *AI degree applications rise 15%, but mixed picture for other computing subjects in 2025*. Available at: https://www.bcs.org/articles-opinion-and-research/ai-degree-applications-rise-15-but-mixed-picture-for-other-computing-subjects-in-2025/ (Accessed: 7 November 2025).

The British Computer Society (no date) *BCS Code of Conduct for members - Ethics for IT professionals*. Available at: https://www.bcs.org/membership-and-registrations/become-a-member/bcs-code-of-conduct/ (Accessed: 18 November 2025).

The National Cyber Security Centre (no date) *Software Security Code of Practice*. Available at: https://www.ncsc.gov.uk/section/software-security-code-of-practice/overview (Accessed: 18 November 2025).

UCAS (2023) *UK 18-year-olds make record number of applications for computing courses*. Available

at: https://www.ucas.com/corporate/news-and-key-documents/news/uk-18-year-olds-make-record-number-applications-computing-courses (Accessed: 7 November 2025).

Videnovik, M., Vold, T., Kiønig, L., Bogdanova, A.M. and Trajkovik, V. (2023) 'Game-based learning in computer science education: a scoping literature review', *International Journal of STEM Education*, 10(1), article 54. Available at: https://doi.org/10.1186/s40594-023-00447-2 (Accessed: 10 November 2025).

Zhan, Z., *He, L., Tong, Y., Liang, X., Guo, S. and Lan, X.* (2022) 'The effectiveness of gamification in programming education: Evidence from a meta-analysis', *Computers and Education: Artificial Intelligence*, 3, article 100096. Available at: https://doi.org/10.1016/j.caeai.2022.100096 (Accessed: 8 November 2025).

Zhang-Kennedy, L. and Chiasson, S. (2021) 'A Systematic Review of Multimedia Tools for Cybersecurity Awareness and Education', *ACM Computing Surveys (CSUR)*, 54(1, Article 12), pp. 1-39. Available at: https://doi.org/10.1145/3427920.

Naming and code style tips for C# scripting in Unity (no date) *Unity*. Available at: https://unity.com/how-to/naming-and-code-style-tips-c-scripting-unity (Accessed: 8 March 2026).

A. Professional, Legal, Ethical and Societal Issues

A.1 Professional Issues

The package created by this project is liable to cause developers to fail to follow good coding practices if the package itself does not follow good coding practices. We will adhere to the BCS Code of Conduct by The British Computer Society (no date) when we design our project and deliver our package, so that users of our package can be assured that they are still following the BCS Code of Conduct.

A.2 Legal Issues

The third example game created by this project (where we modify the game's code to use a GitLab UI and later on modify it to use our package), as well as any packages used by the project, may have copyrights or licenses that need to be adhered to – we will ensure we adhere to these copyrights and licensing through actions such as adding licensing documents to our work where needed. Furthermore, our package may need a license document as well, so we will ensure to properly create a licensing document for our package.

A.3 Ethical Issues

For ethical issues, the package created by this project is liable to cause data leaks if the communication between the package and a GitLab host is compromised. Any security issues like this within the package may leak important information about end-users or the games using the package. It could also leak tokens, which allow access to non-public repositories. To counter this, the Unity method (UnityWebRequest) we use to send HTTP requests by the package uses secure HTTPS connections, which alleviates this issue. We will adhere to the Software Security Code of Practice by The National Cyber Security Centre (no date) to ensure we follow standards for reducing the level of cybersecurity risk in our package (although this one in particular states that it is the minimum required).

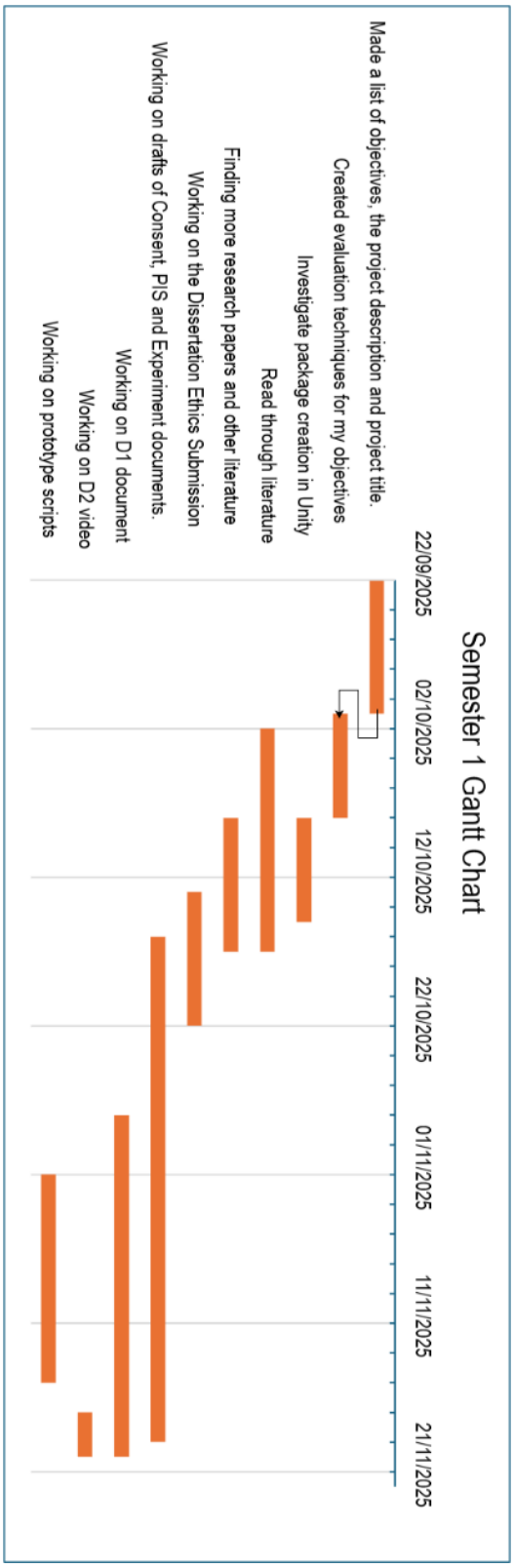
A.4 Societal Issues

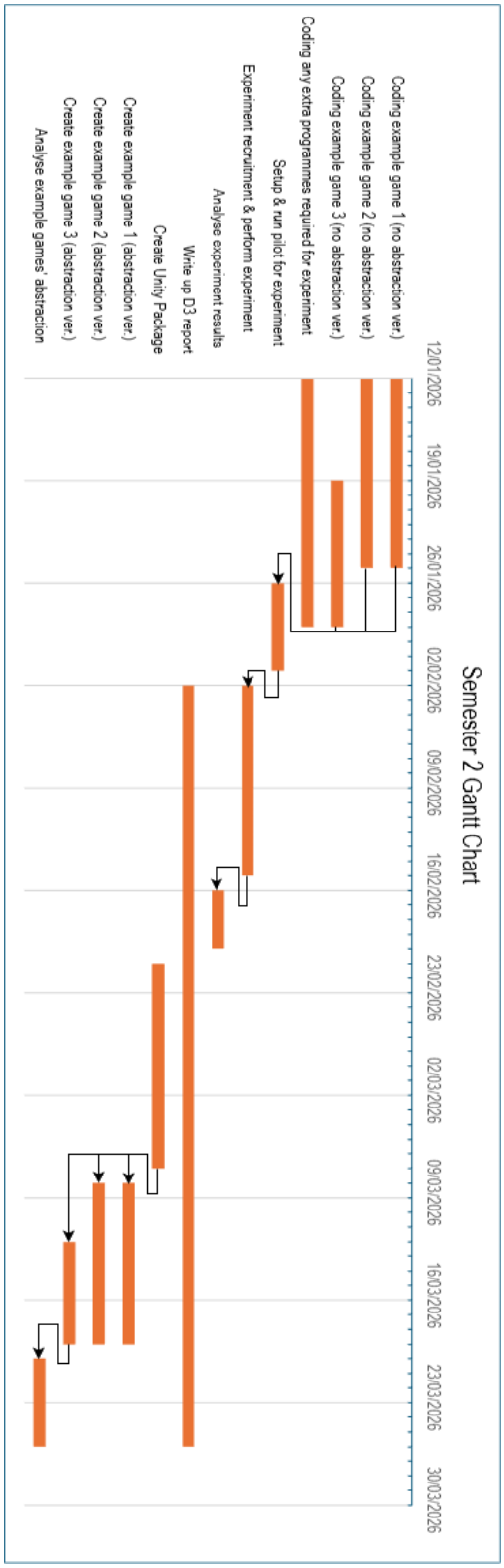
For societal issues, the package created by this project will facilitate the creation of main computer-science-centred serious games. These games will support students in their learning and, since more could be created, will hopefully be able to reach more students than before. The gamification of these programming concepts and the propagation of computing serious games should lead to making computer science education more widespread and accessible.

B. Project Management

Throughout this first section of the project, a Scrum framework was used to manage the project. Weekly meetings with our supervisor acted as sprint reviews and sprint retrospectives, while we planned the next sprint after these meetings. For our prototype, we only used Unity Version Control. We kept track of tasks and meetings using a notebook.

B.1 Gantt Chart


Semester 1 Gantt Chart
22/09/2025
02/10/2025
12/10/2025
22/10/2025
01/11/2025
11/11/2025
21/11/2025
Made a list of objectives, the project description and project title.
Created evaluation techniques for my objectives
Investigate package creation in Unity
Read through literature
Finding more research papers and other literature
Working on the Dissertation Ethics Submission
Working on drafts of Consent, PIS and Experiment documents.
Working on D1 document
Working on D2 video
Working on prototype scripts



B.2 Risks and Mitigation

Risk	Impact	Likelihood / Probability of Occurring	Mitigation Plan
We have fallen ill.	Serious	Low	A Mitigating Circumstances application will be submitted.

Our supervisor leaves.	Serious	Very low	A new supervisor will be appointed.
Storage failure has occurred (e.g computer breaks or is lost).	Catastrophic	Low	Regular backups of code will be done using Unity Version Control. Documents will be saved to our Heriot-Watt OneDrive. More important versions of code and documents will additionally be backed up on GitLab.
Our experiment shows a lack of concrete outcomes.	Serious	Low	We will analyse the general outcome of the results and use it to generate a more generic set of requirements.
Our experiment results prove to be impossible to implement.	Tolerable	Low	We infer the general goals that the results aim to achieve (e.g more versatility) and make a set of requirements that are achievable but still meet these goals.
GitLab API changes.	Tolerable	Very low	Since it will very likely be performing the same function, we simply need to change the code to use the new API request syntax. This shouldn't affect the experiment results because the experiment focuses more on the user's abstraction than the code snippets given.
Unity Packages my package relies on (e.g Newtonsoft for JSON serialization) changes.	Tolerable	Very low	Since they will very likely be performing the same function, we simply need to change the code to use the new package calls. This shouldn't affect the experiment results because the focus is more towards the user's abstraction than the code snippets given.
One or more important requirements emerge later on, which result in a need to completely overhaul all of the work up until that point.	Catastrophic	Low	The work we did for the feasibility section of my D1 document helps to lower the chance of any large, important requirement changes later on. We will also make an effort to ensure our targets and scope are realistic. Additionally, in our Gantt chart plan for semester 2, we allocate many tasks to have some more time than they should probably

			take – this ensures that if this risk were to happen, we would have time to scope out what is feasible in the time we have left.
The time needed for one or more tasks was greatly underestimated.	Serious	Moderate	Always allocate a bit more time to tasks than strictly required. Set tasks to be in parallel when possible, so that if one task cannot be worked on, headway can be made on another – the time saved here should hopefully create extra time to allocate to anything that requires it. Additionally, by using MoSCoW to rank our requirements/goals, we have allowed ourselves to have tasks we can decide not to do in case we run short on time. The most prominent example of this is our 3rd example game, as displayed in our Gantt chart, with the tasks related to the 3rd example game being separated from the tasks related to our 1st and 2nd example games – this highlights the disposable quality of the 3rd example game tasks.
The total number of participants for my experiment is less than three.	Serious	High	More time will be allocated towards recruitment (currently set to 2 weeks).
Our experiment takes longer than expected.	Tolerable	Low	Since the D3 document write-up can be started without the experiment results, do this in conjunction with the experiment if the experiment takes longer than expected. Use the time saved doing this to later catch up on work that will have fallen behind.

C. Generative AI

No Generative AI has been used in this project or any other deliverables.

D. Project Proposal and Research Changes

Requirements FR-2.1.2 and FR-2.1.2.1 were left uncompleted due to a lack of time. However, the functionality of FR-2.1.2 is also covered by FR-2.1.1 which was completed and therefore means we can reduce its MoSCoW ranking to a Won't where we had mistakenly put Could beforehand. Additionally, Our implementation of the autonomous web requests that would delete extra files, replace missing files, and replace changed file contents in a repository (as described in requirement FR-2 and all its sub-requirements), required the use of UniTask in order to successfully call the asynchronous method required.

E. Appendices

In this section, we have placed our draft versions of our consent form, Participant Information Sheet, and experiment questionnaire. We start each new document on a new page to make the documents harder to get mixed up.

Below is our draft consent form.

HERIOT WATT UNIVERSITY

CONSENT FORM

Project title: Unity package for abstracting the development process of games with a GitHub/GitLab UI

Heriot-Watt University attaches high priority to the ethical conduct of research. We therefore ask you to consider the following points before signing this form. Your signature confirms that you are willing to participate in this study, however, signing this form does not commit you to anything you do not wish to do, and you are free to withdraw your participation at any time.

Please tick the initial boxes

- **I confirm** that the purpose of this study was explained to me in sufficient detail and I have had the opportunity to consider the information, to ask questions and have these answered satisfactorily. ☐
- **I confirm** that I am over 16 years old. ☐
- **I understand** that my participation is voluntary and that I am free to withdraw at any time, without giving any reason, and without consequences. ☐
- **I understand** that any data I give will be used in the dissemination of findings and will remain anonymous and that my signed consent form will be securely stored in the HWU One-Drive folder, which is password protected and can be accessed only by the MACS Director of Ethics and HWU Data Protection Officer. ☐
- **I feel** I am well enough, physically and mentally, to complete the tasks as described in the Information Sheet. ☐
- **I agree** to take part in the above study. ☐

Project Student: Helen Pui-Sarn Chang E-mail: hpc2000@hw.ac.uk	Project Supervisor: Manuel Maarek Email: M.Maarek@hw.ac.uk

Name:
Signature:
Date:
Email Address:

I would like to receive information on the results of this study: ☐**es** ☐ **No**

Below is our draft Participant Information Sheet:

INFORMATION SHEET

Project title: Unity package for abstracting the development process of games with a GitHub/GitLab UI

Objectives

For my Hons dissertation, I am investigating the features an optimised GitHub/GitLab UI game development package would have.

The purpose of this information sheet is to inform you of what data we will collect from you and what tasks you will be expected to do during this experiment. It is also to inform you of how you can remove your data or request access to your data from our dataset, as well as other sensitive information pertaining to the experiment.

Data collection
The data that will be collected from you is listed below:

- That you are above the age of 16.
- That you are not a vulnerable member of society.
- Your year and programme of study.
- Observation and feedback from the forms and experiment.

The collected data will be pseudo-anonymised. You will be able to access or delete your data by sending an email requesting to do so to me (hpc2000@hw.ac.uk) or my supervisor (M.Maarek@hw.ac.uk).

Experiment Protocol

You will first be asked to read through a series of code snippets which will each be commented and accompanied by a small description of itself. Once you have understood them, you will be asked to make sure you note down any changes or updates in your ideas or approach in a different style or notation during the rest of the experiment. This should take 5-20 minutes.

You will then be asked to create a list of pseudocode method calls designed to abstract the code snippets aimed at the purpose of being a GitHub/GitLab-UI-having game development package. For each method call, you will also be asked to include an explanation of the parameters of the method, what the method does, any extra functionality you envisioned for the method and any additional information on why you chose to make the method this way. This should take 20 minutes.

Once you have completed that, you will be asked to recreate each of the code snippets you saw in the first segment of the experiment using your abstract method calls. You will be allowed to go back and edit previous answers if you wish to. This should take 10 minutes.

To ensure the safety of collected data, all data collected will be stored on the Heriot-Watt University IT Systems (more specifically on the Heriot-Watt One Drive).

If you wish to participate in this experiment, please access the experiment's Microsoft Form consent form link and complete the form. After you have done so, please access the Microsoft Bookings link and book a timeslot for your participation in the experiment. After this, you will simply have to show up at the specified time and location of the booking and participate in the experiment.

Withdraw

As a participant, you have the right to withdraw at any time before, during or after the experiment without any harm to your employment status, salary, student status or grades.

If you wish to sign up, please access the Microsoft Forms link containing the consent form for this experiment and complete the form.

After you have signed the consent form, if you wish to take part in the experiment, then please follow the instructions explaining how to take part at the bottom of the Experiment Protocol section.

Please note that your signed consent form will be securely stored in the HWU One-Drive folder, which is password protected and can be accessed only by the MACS Director of Ethics and HWU Data Protection Officer.

For additional details or to request the deletion of your data, you can reach out to hpc2000@hw.ac.uk, or you can contact the project supervisor via M.Maarek@hw.ac.uk.

NOTE: Please find below further information on Heriot-Watt Data Protection

1) Data Protection Officer contact details: dataprotection@hw.ac.uk

2) Here is the link to the Heriot-Watt University Privacy Notice for Research Participants: https://www.hw.ac.uk/uk/services/docs/information-governance/PrivacyNoticeResearch-V4Finalversion.pdf

3) Data controller is Heriot-Watt University

Below is our draft experiment questionnaire:

Note: For each of the tasks in this sheet, you may find you want to go back and change your previous answers. Feel free to change your answers to previous questions at any point in the experiment, but please ensure answers to all the following questions are consistent with your changed answer.
Task 1) Please answer the following question:
Which programme and year of your programme are you currently in? (e.g Computer Science 3rd Year)

Task 2) For the following Likert-scale questions, please select the option that most closely applies to you:
I am experienced in the area of games programming.

Strongly Agree	Agree	Neutral	Disagree	Strongly Disagree

I am experienced in the area of API programming.

Strongly Agree	Agree	Neutral	Disagree	Strongly Disagree

I am experienced in the area of Dev Ops.

Strongly Agree	Agree	Neutral	Disagree	Strongly Disagree

Please comment below any experiences relating to the Likert-scale questions that you would like to share. If there are none, please leave this blank.

Task 3) Please read and understand the following code snippets. Comments and Descriptions have been provided to assist you in this task.
<Insert Code Snippets here>
Task 4)
From Task 3 onward, it is requested that you note down in a notable style/notation easily differentiable from the answers you will be providing any changes/upgrades to your ideas, approaches, or answers, along with any reasoning for those changes/upgrades. Please make these additional notes above or below the change/upgrade you make.

You may change this later if you wish to do so, but please keep the style/notation you use for your extra notes consistent with your answer for this question. Please type out the style/notation you will use for these extra notes below.

Task 5) Please create a list of pseudocode method calls to abstract the code snippets you saw previously. Please centre the use of these method calls towards the development of games that use GitHub/GitLab as a User Interface. Please feel free to change/upgrade your answers here as long as they stay consistent with your answers in Task 4.
For each method call you create, please add underneath the method call:

- An explanation of the method call's parameters
- A description of the related method's behaviour/purpose

- Any extra functionality you envisioned for the related method
- Any extra information as to why you designed the method call/related method this way

Task 6) Underneath each header below, please recreate the code snippet with the same name using your abstract method calls from Task 3.

Consent Form

Helen Chang's Dissertation Experiment's Consent Form

* Required

* This form will record your name, please fill your name.

1. **Project Student:** Helen Pui-Sarn Chang
   E-mail: hpc2000@hw.ac.uk

   Project Supervisor: Manuel Maarek
   Email: M.Maarek@hw.ac.uk

   Please note that the Participant Information Sheet containing this experiment's details is provided within the same email/message in which you received the link to this consent form. You can also access it through this link: https://heriotwatt-my.sharepoint.com/:b:/g/personal/hpc2000_hw_ac_uk/IQBU_6eiOX3JSodczdIuGXnIAVBFVLqU6sPUz7fROsXxphg?e=WtAjh3

2. ***Project title:*** *Unity package for abstracting the development process of games with a GitHub/GitLab UI*

   Heriot-Watt University attaches high priority to the ethical conduct of research. We therefore ask you to consider the following points before signing this form. Your signature confirms that you are willing to participate in this study, however, signing this form does not commit you to anything you do not wish to do, and you are free to withdraw your participation at any time.

   Please tick the initial boxes

3. *

   ◯ **I confirm** that the purpose of this study was explained to me in sufficient detail and I have had the opportunity to consider the information, to ask questions and have these answered satisfactorily.

4. *

   ◯ **I confirm** that I am over 16 years old.

5. *

◯ **I understand** that my participation is voluntary and that I am free to withdraw at any time, without giving any reason, and without consequences.

6. *

◯ **I understand** that any data I give will be used in the dissemination of findings and will remain anonymous and that my signed consent form will be securely stored in the HWU One-Drive folder, which is password protected and can be accessed only by the MACS Director of Ethics and HWU Data Protection Officer.

7. *

◯ **I understand** that any data I give will be anonymised to remove any confidential/personal information.

8. *

◯ **I feel** I am well enough, physically and mentally, to complete the tasks as described in the Information Sheet.

9. *

◯ **I agree** to take part in the above study.

10. Signature *

11. I would like to receive information on the results of this study: *

◯ Yes

◯ No

Below is the actual questionnaire used in the experiment.

Note: For each of the tasks in this sheet, you may find you want to go back and change your previous answers. Feel free to change your answers to previous questions at any point in the experiment, but please ensure answers to all the following questions are consistent with your changed answer.

Task 1 – Year and Programme

Please answer the following question:
Which programme and year of your programme are you currently in? (e.g Computer Science 3rd Year)

Task 2 - Experience

For the following Likert-scale questions, please select the option that most closely applies to you:

I am experienced in the area of games programming.

Strongly Agree	Agree	Neutral	Disagree	Strongly Disagree

I am experienced in the area of API programming.

Strongly Agree	Agree	Neutral	Disagree	Strongly Disagree

I am experienced in the area of Dev Ops.

Strongly Agree	Agree	Neutral	Disagree	Strongly Disagree

Please comment below any experiences relating to the Likert-scale questions that you would like to share. If there are none, please leave this blank.

Task 3 – Understanding the code

In your participant folder, you have been provided with two files: one named ExperimentTask3Participantx and one named InputParameterSheetParticipantx where x is your participant number.

Please read and familiarise yourself with the code found in the ExperimentTask3Participantx file. Comments, output parameters, and Descriptions have been provided within to assist you in this task. Input parameters have been provided in the InputParameterSheetx file to provide additional assistance.

Notes:

- The input parameters can be found separately at the top of the document
- Output parameters can be found at the bottom of each script in the form of data structures. These data structures are repeated across scripts.
- Game 3 has a lot of scripts that aren't included in the code we give you to read. The relevant functions are explained in comments.

Task 4 – Picking a notation

From task 5 onwards, we ask that you note down any changes to your ideas, approaches, or answers, along with the reasoning for said changes. Please make these additional notes above or below the change you make in a style/notation that is easily differentiable from the answers you will be providing.

You may change this later if you wish to do so, but please keep the style/notation you use for your extra notes consistent with your answer for this question.
Please type out the style/notation you will use for these extra notes below (e.g highlighting):

Task 5 – Making an abstraction

Please create a list of pseudocode method calls to abstract the code snippets you saw previously. Please centre the use of these method calls towards the development of games that use GitHub/GitLab as a User Interface. Please feel free to change/upgrade your answers here as long as they stay consistent with your answers in Task 4.

For each method call you create, please add underneath the method call:

- An explanation of the method call's parameters
- A description of the related method's behaviour/purpose
- Any extra functionality you envisioned for the related method
- Any extra information as to why you designed the method call/related method this way

Please write your method calls below:

Task 6 – Recreating the games' code

Within the ExperimentTask6Participantx file where x is your participant number, you have been provided with a copy of the game code you read in task 3. Please recreate the game code by modifying the code in ExperimentTask6Participantx to use the method calls you created in task 5.

Your code does not have to be functional. You are only pseudocoding the method calls into the code.

Please Note: The experiment's ExperimentTask3Participantx sheet and ExperimentTask6Participantx sheets have been deemed too long to fit here (As they are both around 51 pages long) as well as the InputParameterSheetParticipantx sheet which

is 10 pages long but can be found in the logbook related GitLab repository here: https://gitlab-student.macs.hw.ac.uk/hpc2000/helenchangdissertationrepo

Below is my intermediate stage analysis of my experiment results – congregating method call names and facts together like this made it easier to analyse.

Participant 1

4th year MEng SE

Experienced in all areas (above average) – specific experience in 3rd year group project

Participant did not know how he would make his package about games

Used Type? To show nullable parameters – not passed = null

Parameters from the input API (pretty much 1-1)

Status code abstracted (succeed = Success enum e.g 200 == success) + error code

Methods:

1) Constructor() Method to create an object of gitlab repo for interacting with that repo in specific
2) DeleteRequest() Method to delete a file/directory given a filepath
3) CreateRequest() Method to create a file
4) GetFile() Method to get a file
5) UpdateFileRequest() Method to update a file
6) GetRepositoryTreeRequest() Method to get repo tree
7) GetRepositoryCommitsRequest() Method to get repo commits
8) GetCommitRequest() Method to get a specific commit request
9) GetSingleUserRequest() Method to get current user (returns user object)
10) GetPipelineRequest() Method to get a pipelineor
11) CreateIssueRequest() Method to create an issue
12) CreateCommit() Method to create a commit

Methods handle length checking , etc. internally.

Methods work like: if success do this code given by user else if fail then do fail code

They return the objects I made

He made the user choose branches for filepaths e.g MAIN

5 changes mostly about parameters

Participant 2

MEng 4th year SE

Experienced in all areas (above average) except API (neutral)

Methods take in mainURL + project code

Methods:

1) checkForFile() A method to check if a file exists in a repo (given a file name, is it ANYWHERE in the repo?) returns bool
2) CheckFileAtLocation() A method to check if a file is at a certain filepath returns bool

3 changes mostly realisations

Participant 4

BSc 3rd year Comp Sci

Above average experience in API + games

Below average experience in Dev Ops

Methods:

1) webRequest() One method that does everything – passed in an OUT parameter where JSON will be saved. Passed in success codes to check for. Checked for repeats of requests to do a commit request instead (he said this version would be messier than his original)
   He seemed to expect users to deserialize the returned results themselves to make returning easier as it would just have to be a string.
   User seems to construct the URL used but function decides GET, POST, DELETE, PUT etc.
   Returns true/false for success of the request
2) validateInputs() One method that takes in URL, projectNUM, and token and checks they're valid types.
3) elementOverlap() One method to check for the existence of two strings in one file path.

5 changes mostly extra improvements.

Participant 5

BSc Computer Science 4th year

Above average all

Methods:

1) Used my OnSubmitRepo()/SubmitRepo()
2) OnSubmitHouse()
3) GetFileRequest()
4) RetrieveACommitRequest()
5) testreportRequest() to retrieve test report

Although he did it incorrectly, when trying to summarise the code (as he summarised instead of abstracted) he gravitated towards submitRepo() method, requestSend() method, validateRepo() etc. Aka. Big functions covering a lot of functionality.

6 changes – half misunderstanding task, half misunderstanding code

Participant 6

MSc AI 1st year

Above average experience in all but API where he had a large amount of experience.

Methods:

1) CheckGitStatus() Method that takes in token + url and checks it can valid get, post etc. takes care of invalid input (type, length)
2) FetchCommits() Method to fetch all commits of a project (takes in project url, etc.) + can take in dateTimeStamp parameter to fetch commits up to a certain date. He has a apiVersion parameter to change api version from v4 in requests. Returns commits object list.
3) FetchCommitDetails() Method takes in repo url stuff + commit id to get a single commit's details
4) GetRepoTree() takes in repo stuff to get repo tree + optional REF parameter to get from a commit/branch
5) GetFileContentsFromPath() takes in repo url stuff + file path to get file contents. Optional REF to get from a commit/branch. Returned ONLY the string
6) GetPipelineReport() takes in repo url stuff + pipeline id to get a pipeline object or if report_type given then returns a pipeline report object instead. (seems to have gotten the "/test_report" end of the url call confused.)
7) GetUser() gets current user when given host name + token. Options apiVersion parameter
8) DeleteFile() deletes a file when given repo url stuff + commit message and branch parameter.
9) CreateFile() creates a file in a repo when given repo url stuff + path + commit message + branch parameter + contents. Seemed to use this for update file as well in task 6 but this may have been by accident.

13 changes most of which is adding new methods or new arguments

Below is a copy of the requirements table from Section 3.2. The requirement descriptions have been replaced by a description of how and why we consider a requirement to be met or not met. The priority column is colour-coded in terms of whether they have been met: Green is for met, red is for not met, and grey is for was not required (was a Won't priority).

ID	Implementation	Priority
FR-1	The sub-requirements are completed so this may be considered completed as well.	
FR-1.1	The Package takes in GitLab hostUrl parameters and so can access both hosts.	Must
FR-1.2	GetFile, GetCommit, etc. are methods in the package which retrieve data for users.	Must
FR-1.2.1	Folder names, file names, and repository structure can all be retrieved using GetRepositoryTrees and file contents can be retrieved using GetFile.	Must
FR-1.2.2	GetPipelineTestReport allows users access to the unit test report of a pipeline.	Could
FR-1.2.3	User profiles can be retrieved using GetUser and the update time of files can be accessed through using GetCommit on the last commit from a GetFile request.	Could
FR-1.3	Users can sends web requests to GitLab through the use of methods which abstract the process. Some of these methods (such as CreateFile, UpdateFile, etc.) take data to send to a GitLab host.	Must
FR-1.3.1	The package uses a custom RequestQueue class object to ensure only one method can send web requests at any time. By preventing requests from being sent simultaneously, any errors caused by requests being sent too closely are avoided.	Must
FR-1.3.2	New files and directories can be created using CreateFile (directories through adding more sections to the filepath when creating a file) and file contents can be updated through UpdateFile. Directory contents can be updated through using DeleteFile and CreateFile to change the file names.	Must
FR-1.3.3	The DeleteFile method can delete files and directories from a repository.	Must
FR-1.4	UpdateFile has a bool parameter EncodeContents which allows users to optionally encode the new contents of the file.	Must
FR-1.5	The package can interact with repositories of all protection types as long as the provided project access token has the correct permissions.	Must
FR-1.6	As the package takes in a new hostURL, projectCode, and project access token every time a method is called, a new repository can always be accessed.	Must
FR-1.7	CreateFile and UpdateFile both take in strings as the content parameter and any indentation, spacing, etc. will be uploaded to the repository. This means in normal files it will keep the indentation, spacing, etc. and in markdown files it will display according to the syntax in the uploaded string.	Must
FR-2	As the only sub-requirements not met (FR-2.1.2 and FR-2.1.2.1) either duplicate the functionality of other sub-requirements or are not important to the overall goal of allowing users to call methods which continuously check for and revert changes to a repository – therefore we can consider this requirement complete overall.	
FR-2.1	CheckChanges methods returns a list of files added, a list of files deleted, and a list of files whose contents have changed (unless they have been deleted).	Must

	CheckFileChanged returns a string stating if the file at the given file path has contents that differ from the given original file contents.	
FR-2.1.1	This functionality can be found in AutonomousDeleteExtra and AutonomousReplaceMissing where when activated they will continuously request files contents, repository structure etc. and delete extra files or replace files and file contents respectively.	Must
FR-2.1.2	Due to a lack of time for implementation, and the functionality duplicating that of 2.1.1 – it was decided that the priority for this requirement was low and therefore this was not implemented.	Could
FR-2.1.2.1	Due to 2.1.2 not being implemented and a lack of time, this was not implemented.	Could
FR-2.2	An AutonomousDeleteExtra object which asynchronously deletes extra files (extra directories will automatically be deleted when the extra files inside them are deleted) can be created (and automatically started) using createAutonomousDelete.	Must
FR-2.3	The AutonomousReplaceMissing object asynchronously replaces missing files (which would automatically replace missing directories when the files inside are replaced) and update files to have their original contents can be created (and automatically started) using createAutonomousReplace. The original contents structure can be edited to use	Must
FR-3	The sub-requirements are completed so this may be considered completed as well.	
FR-3.1	The RequestQueue can have its inner queue be accessed as a string using its getQueueAsString method and access as the actual object queue using its getQueue method.	Should
FR-3.2	The RequestQueue has a Clear method which clears itself. Methods detect when they have been removed from the request queue when waiting for permission, skip doing any work, and return a Response.Cancelled enum.	Should
NFR-1	As the sub-requirements for this section have been completed, we can say that the package is easier to use than the users implementing the functionality themselves.	Must
NFR-1.1	The package's method calls abstracting web requests put together the URIs for the requests using the parameters provided by the user (hostUrl, projectCode, etc.).	Must
NFR-1.2	The package's method calls abstracting web requests remove the need for the user to code and send their own web requests, reducing it to one line.	Must
NFR-2	As seen in Section 6.3, we analysed the experiment results to make intuitive method calls. Sub-requirements are complete so this can be considered complete.	Must
NFR-2.1	In Section 6.3, we identified a naming style (Verb before noun) that lets us easily differentiate method names but still keep them understandable whilst letting us give methods with similar functionalities similar names.	Must
NFR-2.1.1	Package methods with similar functionality have had their parameter names and order made as similar as possible. Package methods abstracting web requests have also had a standardised parameter order made.	Should
NFR-2.2	Parameters have been given short names that should be easily understood.	Must
NFR-2.2.1	Parameter names are repeated across methods where applicable and an overall standard parameter order has been made for methods in the package.	Must

NFR-2.3	The package's methods return an Response enum which has 4 values: Success, Error, InvalidDetails, and Cancelled. All of which are easily understood.	Must
NFR-2.3.1	All web request methods in the package have an overload which returns a ReponseObj object in an out parameter. ResponseObj objects contain a response Message (often containing the download handler text from a request which would hold any errors or holding custom text describing what happened e.g., "invalid details were given") and the response code from the request (if a method has multiple requests this would be a success or would return the information from the point of failure)	Should
NFR-3	The sub-requirements are completed so this may be considered completed as well.	Must
NFR-3.1	The package uses the same methods to parse, encode, and decode strings as well as using the same encoder object to do so when such an object is needed.	Must
NFR-3.2	All methods will validate the given parameters before making and sending a request (	Must
NFR-3.3	During input validation the length of the given parameters are checked against public size parameters of the package – if the given parameters are larger than these sizes then the method stops and Reponse.InvalidDetails is returned.	Must
NFR-3.4	Encoding of parts of web requests is done right before URI length checking (as it would be part of the URI) and then the web request is sent immediately after.	Must
NFR-3.5	An effort has been made to remove references to variables, objects, etc. where possible to ensure it gets cleaned up during garbage collection. Web requests are also enclosed in a Using statement to ensure it gets cleaned up once used.	Must
NFR-3.6	We have previously stated this would not be done.	Won't
NFR-3.6.1	We have previously stated this would not be done.	Won't
NFR-4	A GitLabUnityUI.md file is found in the documentation~ folder in the package. As the file contains all the things in the following subsections as well as the recommended sections from the Unity guidance, we consider it complete.	
NFR-4.1	A section about the package's thread safety is featured in the documentation file.	Could
NFR-4.2	A section about the package's error handling is featured in the documentation file.	Should
NFR-4.3	A section about the package's inputs, usage, and return values is featured in the documentation file.	Must